Stat 405 project

Jack

2026-04-07

Data Cleaning

```
library(cmdstanr)
```

```
## This is cmdstanr version 0.9.0
```

```
## - CmdStanR documentation and vignettes: mc-stan.org/cmdstanr
```

```
## - CmdStan path: /Users/jackyip/.cmdstan/cmdstan-2.38.0
```

```
## - CmdStan version: 2.38.0
```

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(ggplot2)
library(posterior)
```

```
## This is posterior version 1.7.0
```

```
##
## Attaching package: 'posterior'
```

```
## The following objects are masked from 'package:stats':
##
##     mad, sd, var
```

```
## The following objects are masked from 'package:base':
##
##     %in%, match
```

```
library(bayesplot)
```

```
## This is bayesplot version 1.15.0
```

```
## - Online documentation and vignettes at mc-stan.org/bayesplot
```

```
## - bayesplot theme set to bayesplot::theme_default()
```

```
##    * Does _not_ affect other ggplot2 plots
```

```
##    * See ?bayesplot_theme_set for details on theme setting
```

```
##
## Attaching package: 'bayesplot'
```

```
## The following object is masked from 'package:posterior':
##
##     rhat
```

```
library(coda)
```

```
## Warning: package 'coda' was built under R version 4.3.3
```

```
library(loo)
```

```
## Warning: package 'loo' was built under R version 4.3.3
```

```
## This is loo version 2.8.0
```

```
## - Online documentation and vignettes at mc-stan.org/loo
```

```
## - As of v2.0.0 loo defaults to 1 core but we recommend using as many as possible. Use
the 'cores' argument or set options(mc.cores = NUM_CORES) for an entire session.
```

```
##
## Attaching package: 'loo'
```

```
## The following object is masked from 'package:posterior':
##
##     gpdfit
```

```
library(tidyr)

housing_data <- read.csv("~/Downloads/melb_data.csv")
part1_data <- housing_data %>%
  mutate(
    log_price = log(Price),
    distance_z = as.numeric(scale(Distance)),
    buildingarea_z = as.numeric(scale(BuildingArea))
  ) %>%
  filter(
    !is.na(log_price),
    !is.na(distance_z),
    !is.na(buildingarea_z),
    !is.na(Regionname)
  )

part1_data$region_id <- as.integer(as.factor(part1_data$Regionname))

stan_data_model1 <- list(
  N = nrow(part1_data),
  y = part1_data$log_price,
  distance = part1_data$distance_z,
  buildingarea = part1_data$buildingarea_z
)

stan_data_model2 <- list(
  N = nrow(part1_data),
  R = length(unique(part1_data$region_id)),
  y = part1_data$log_price,
  distance = part1_data$distance_z,
  buildingarea = part1_data$buildingarea_z,
  region = part1_data$region_id
)
```

Model 1: Stan model (method 1)

fitting for STAN Model

```
mod1 <- cmdstan_model("Model1.stan")

fit_m1 <- mod1$sample(
  data = stan_data_model1,
  chains = 4,
  parallel_chains = 4,
  iter_warmup = 1000,
  iter_sampling = 1000,
  seed = 123,
  init = 0,
  refresh = 0,
  adapt_delta = 0.95,
  max_treedepth = 12
)
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 finished in 11.1 seconds.
## Chain 3 finished in 11.3 seconds.
## Chain 2 finished in 12.8 seconds.
## Chain 4 finished in 14.1 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 12.3 seconds.
## Total execution time: 14.2 seconds.
```

posterior summaries

```
summ_m1 <- fit_m1$summary(variables = c("beta0", "beta1", "beta2", "sigma")) %>%
  as.data.frame() %>%
  dplyr::select(variable, mean, median, sd, q5, q95, rhat, ess_bulk, ess_tail)

summ_m1
```

```
##   variable        mean      median          sd          q5         q95     rhat
## 1    beta0 13.73941273 13.73921300 0.006139360 13.72925830 13.74960830 1.005012
## 2    beta1 -0.09291254 -0.09300522 0.006276566 -0.10294222 -0.08262237 1.001508
## 3    beta2  0.06173576  0.06167671 0.006555387  0.05105858  0.07252791 1.000429
## 4    sigma  0.53106841  0.53102125 0.004452695  0.52371207  0.53844117 1.001245
##   ess_bulk ess_tail
## 1 4919.469 2946.141
## 2 5556.926 2903.859
## 3 5112.316 2274.807
## 4 4517.947 2801.776
```

Stan sampler diagnostics 1 (Checks divergences and treedepth)

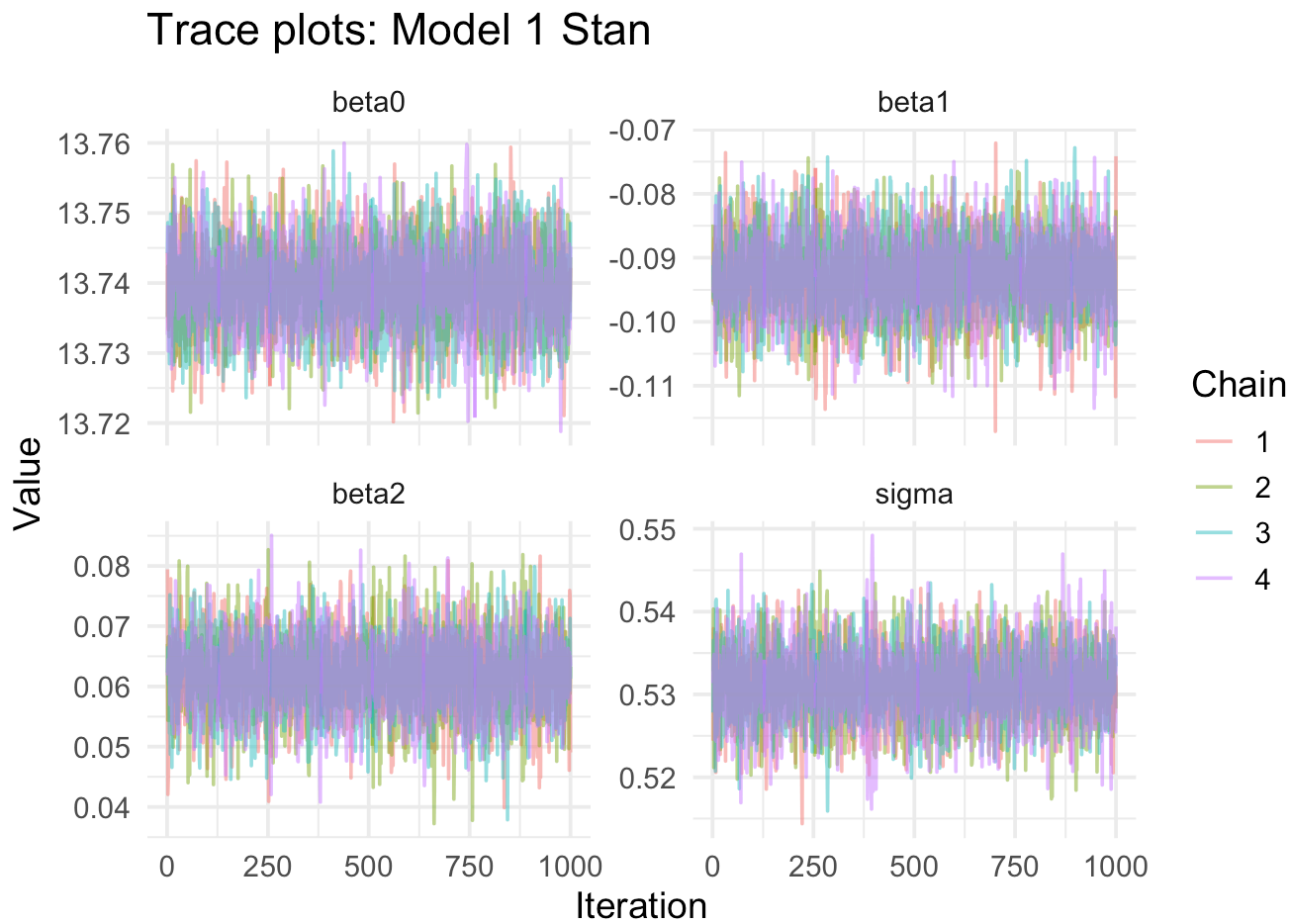
```
check_sampler_issues <- function(fit, treedepth_limit = 12) {
  sdg <- fit$sampler_diagnostics()
  data.frame(
    divergent = sum(sdg[, , "divergent__"]),
    hit_max_treedepth = sum(sdg[, , "treedepth__"] >= treedepth_limit),
    max_treedepth_seen = max(sdg[, , "treedepth__"])
  )
}

check_sampler_issues(fit_m1, 12)
```

```
##   divergent hit_max_treedepth max_treedepth_seen
## 1         0                 0                  3
```

Trace plot

```
fit_m1$draws(variables = c("beta0", "beta1", "beta2", "sigma"), format = "draws_df") %>%
  dplyr::select(.chain, .iteration, beta0, beta1, beta2, sigma) %>%
  pivot_longer(cols = c(beta0, beta1, beta2, sigma), names_to = "parameter", values_to =
"value") %>%
  ggplot(aes(x = .iteration, y = value, colour = factor(.chain))) +
  geom_line(alpha = 0.5) +
  facet_wrap(~ parameter, scales = "free_y", ncol = 2) +
  labs(title = "Trace plots: Model 1 Stan", x = "Iteration", y = "Value", colour = "Chai
n") +
  theme_minimal(base_size = 14)
```

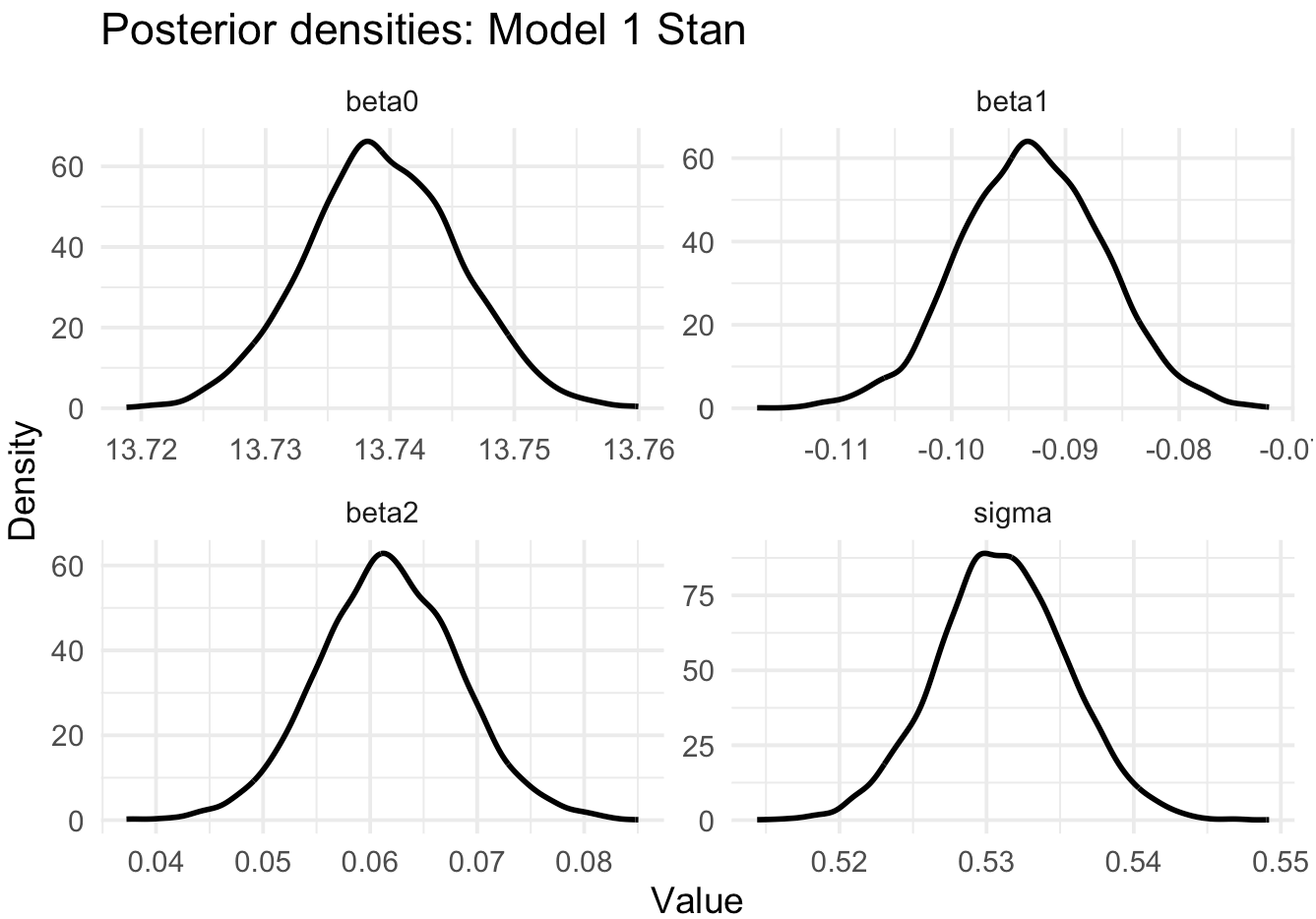
```
## Warning: Dropping 'draws_df' class as required metadata was removed.
```



Posterior density plot (mixing diagnostics)

```
fit_m1$draws(variables = c("beta0", "beta1", "beta2", "sigma"), format = "draws_df") %>%
  dplyr::select(beta0, beta1, beta2, sigma) %>%
  pivot_longer(cols = everything(), names_to = "parameter", values_to = "value") %>%
  ggplot(aes(x = value)) +
  geom_density(linewidth = 1) +
  facet_wrap(~ parameter, scales = "free", ncol = 2) +
  labs(title = "Posterior densities: Model 1 Stan", x = "Value", y = "Density") +
  theme_minimal(base_size = 14)
```

```
## Warning: Dropping 'draws_df' class as required metadata was removed.
```



MCMC convergence diagnostics

warm up draws

```
m1_draws <- fit_m1$draws(
  variables = c("beta0", "beta1", "beta2", "sigma"),
  format = "draws_df"
) %>%
  as.data.frame() %>%
  dplyr::select(.chain, beta0, beta1, beta2, sigma)
```

split them into 4 chains from draw

```
m1_chain_list <- split(m1_draws, m1_draws$.chain)

m1_chain_list <- lapply(m1_chain_list, function(df) {
  coda::mcmc(df[, c("beta0", "beta1", "beta2", "sigma")])
})

m1_mcmc_list <- coda::mcmc.list(m1_chain_list)
```

Gelman-Rubin diagnostic (convergence diagnostics) + ESS

```
coda::gelman.diag(m1_mcmc_list, autoburnin = FALSE)
```

```
## Potential scale reduction factors:
##
##       Point est. Upper C.I.
## beta0          1          1
## beta1          1          1
## beta2          1          1
## sigma          1          1
##
## Multivariate psrf
##
## 1
```

```
coda::effectiveSize(m1_mcmc_list)
```

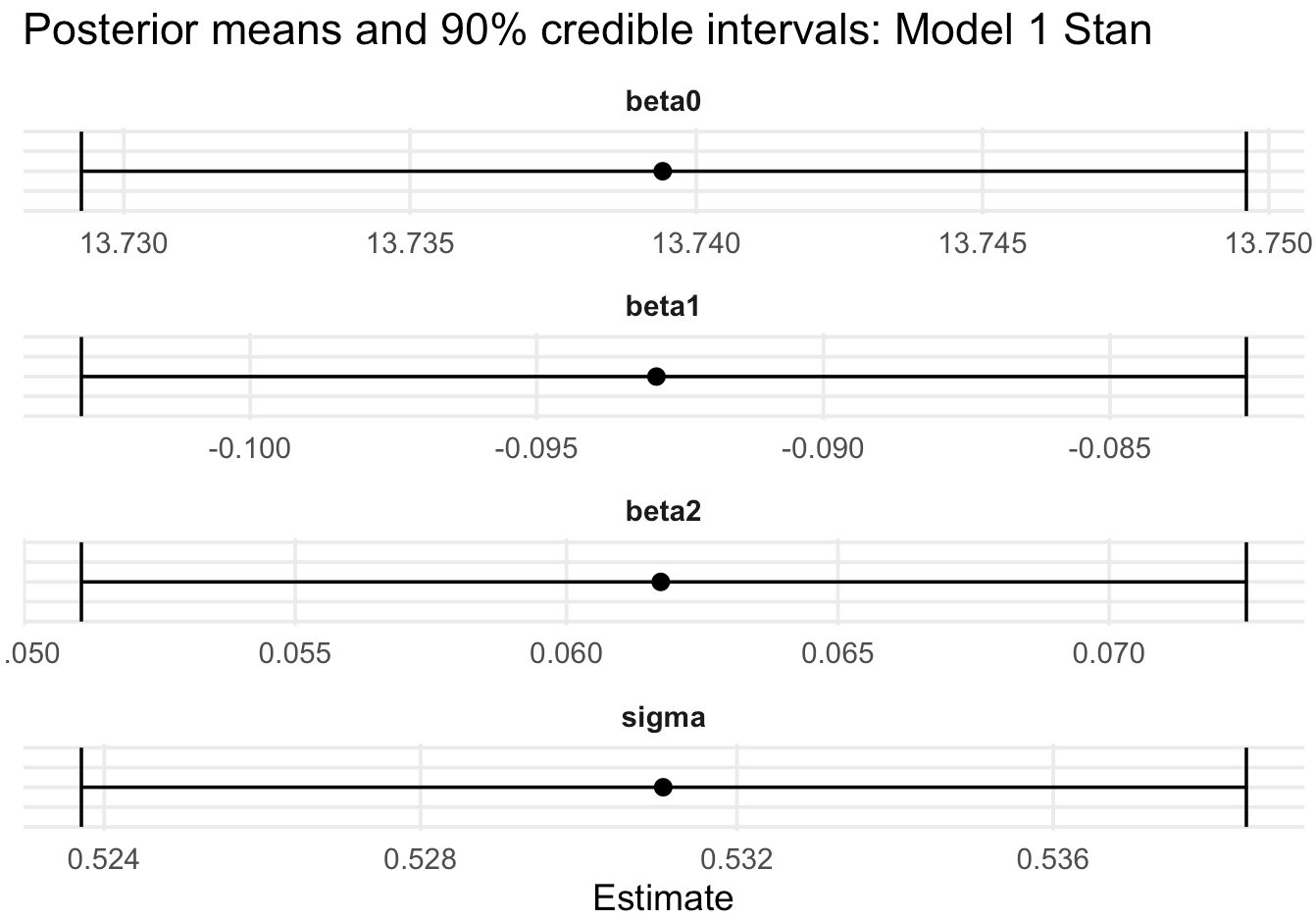
```
##    beta0    beta1    beta2    sigma
## 4946.231 5515.199 4846.869 4544.703
```

Credible intervals for Model 1 Stan

```
summ_m1_plot <- summ_m1 %>%
  filter(variable %in% c("beta0", "beta1", "beta2", "sigma"))

ggplot(summ_m1_plot, aes(x = mean, y = 1)) +
  geom_point(size = 2.5) +
  geom_errorbarh(aes(xmin = q5, xmax = q95), height = 0.08) +
  facet_wrap(~ variable, scales = "free_x", ncol = 1) +
  labs(
    title = "Posterior means and 90% credible intervals: Model 1 Stan",
    x = "Estimate",
    y = NULL
  ) +
  theme_minimal(base_size = 14) +
  theme(
    axis.text.y = element_blank(),
    axis.ticks.y = element_blank(),
    panel.grid.minor = element_blank(),
    strip.text = element_text(face = "bold")
  )
```

```
## Warning: `geom_errorbarh()` was deprecated in ggplot2 4.0.0.
## ℹ Please use the `orientation` argument of `geom_errorbar()` instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
## `height` was translated to `width`.
```



Prior Sensitivity check - robustness check on prior choices

fitting

```
mod1_sens <- cmdstan_model("model1_sensitivity.stan")

prior_settings_m1 <- list(
  baseline = list(beta0_sd = 10, beta_sd = 1, sigma_rate = 1),
  diffuse  = list(beta0_sd = 20, beta_sd = 2, sigma_rate = 0.5),
  regular  = list(beta0_sd = 5,  beta_sd = 0.5, sigma_rate = 2)
)

fits_m1_sens <- lapply(prior_settings_m1, function(pr) {
  mod1_sens$sample(
    data = c(
      list(
        N = stan_data_model1$N,
        y = stan_data_model1$y,
        x_dist = stan_data_model1$distance,
        x_area = stan_data_model1$buildingarea
      ),
      pr
    ),
    chains = 4,
    parallel_chains = 4,
    iter_warmup = 1000,
    iter_sampling = 1000,
    seed = 123,
    refresh = 0
  )
})
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 finished in 6.6 seconds.
## Chain 4 finished in 6.6 seconds.
## Chain 2 finished in 6.7 seconds.
## Chain 3 finished in 7.1 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 6.8 seconds.
## Total execution time: 7.2 seconds.
##
## Running MCMC with 4 parallel chains...
##
## Chain 1 finished in 6.6 seconds.
## Chain 3 finished in 6.6 seconds.
## Chain 4 finished in 7.2 seconds.
## Chain 2 finished in 7.2 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 6.9 seconds.
## Total execution time: 7.3 seconds.
##
## Running MCMC with 4 parallel chains...
##
## Chain 2 finished in 8.5 seconds.
## Chain 3 finished in 8.6 seconds.
## Chain 4 finished in 8.8 seconds.
## Chain 1 finished in 9.0 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 8.7 seconds.
## Total execution time: 9.1 seconds.
```

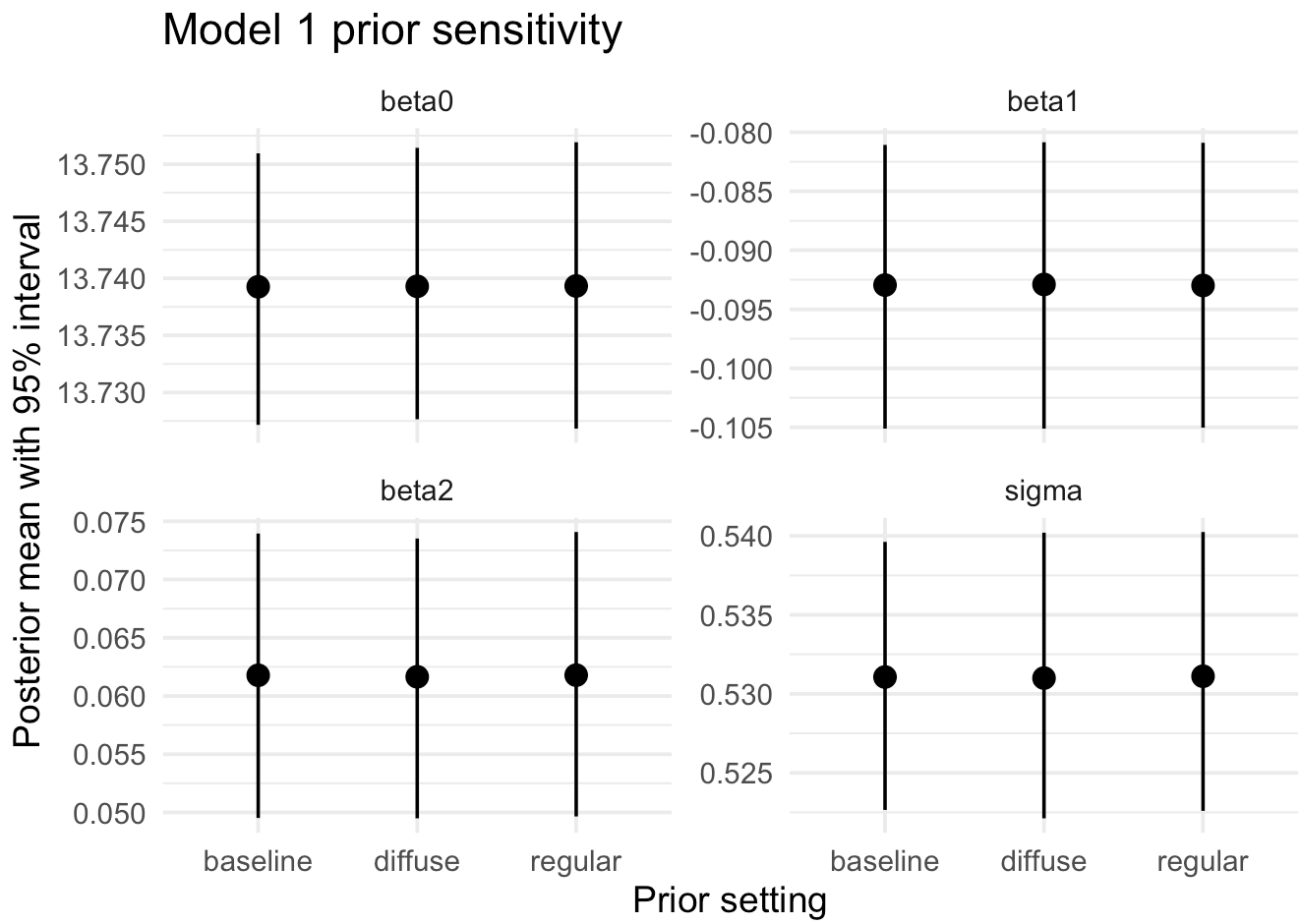
sensitivity table (just a numeric comparison)

```
summarize_fit_m1 <- function(fit, prior_name) {
  draws <- fit$draws(variables = c("beta0", "beta1", "beta2", "sigma"), format = "draws_
df")
  dplyr::bind_rows(
    data.frame(prior = prior_name, parameter = "beta0", mean = mean(draws$beta0), q2.5 =
quantile(draws$beta0, 0.025), q97.5 = quantile(draws$beta0, 0.975)),
    data.frame(prior = prior_name, parameter = "beta1", mean = mean(draws$beta1), q2.5 =
quantile(draws$beta1, 0.025), q97.5 = quantile(draws$beta1, 0.975)),
    data.frame(prior = prior_name, parameter = "beta2", mean = mean(draws$beta2), q2.5 =
quantile(draws$beta2, 0.025), q97.5 = quantile(draws$beta2, 0.975)),
    data.frame(prior = prior_name, parameter = "sigma", mean = mean(draws$sigma), q2.5 =
quantile(draws$sigma, 0.025), q97.5 = quantile(draws$sigma, 0.975))
  )
}

sens_table_m1 <- dplyr::bind_rows(
  summarize_fit_m1(fits_m1_sens$baseline, "baseline"),
  summarize_fit_m1(fits_m1_sens$diffuse, "diffuse"),
  summarize_fit_m1(fits_m1_sens$regular, "regular")
)

sens_table_m1
```

```
##              prior parameter        mean        q2.5       q97.5
## 2.5%...1  baseline     beta0 13.73925948 13.72715770 13.75095115
## 2.5%...2  baseline     beta1 -0.09294977 -0.10510790 -0.08107133
## 2.5%...3  baseline     beta2  0.06177745  0.04952635  0.07395435
## 2.5%...4  baseline     sigma  0.53106662  0.52265500  0.53962361
## 2.5%...5   diffuse     beta0 13.73930046 13.72764657 13.75143915
## 2.5%...6   diffuse     beta1 -0.09289619 -0.10510979 -0.08084829
## 2.5%...7   diffuse     beta2  0.06164919  0.04948932  0.07352390
## 2.5%...8   diffuse     sigma  0.53099960  0.52211754  0.54020308
## 2.5%...9   regular     beta0 13.73933000 13.72682960 13.75192708
## 2.5%...10  regular     beta1 -0.09298413 -0.10502744 -0.08089224
## 2.5%...11  regular     beta2  0.06178984  0.04965204  0.07408357
## 2.5%...12  regular     sigma  0.53111459  0.52259387  0.54024698
```

graph

```
ggplot(sens_table_m1, aes(x = prior, y = mean, ymin = q2.5, ymax = q97.5)) +
  geom_pointrange() +
  facet_wrap(~ parameter, scales = "free_y") +
  labs(title = "Model 1 prior sensitivity", x = "Prior setting", y = "Posterior mean wit
h 95% interval") +
  theme_minimal(base_size = 14)
```



Model 1 - Custom MH (method 2)

Define model 1 with our target log posterior

```
log_posterior_model1 <- function(theta, y, distance, buildingarea) {
  beta0 <- theta[1]
  beta1 <- theta[2]
  beta2 <- theta[3]
  eta <- theta[4]
  sigma <- exp(eta)

  mu <- beta0 + beta1 * distance + beta2 * buildingarea

  log_prior <-
    dnorm(beta0, 0, 10, log = TRUE) +
    dnorm(beta1, 0, 1, log = TRUE) +
    dnorm(beta2, 0, 1, log = TRUE) +
    dexp(sigma, rate = 1, log = TRUE) + eta

  log_like <- sum(dnorm(y, mean = mu, sd = sigma, log = TRUE))

  log_prior + log_like
}
```

Create MH update and sampling functions, basically is a setup for custom posterior sample

```
mh_step_with_lp <- function(current, current_lp, proposal_sds, y, distance, buildingare
a, lp_fun) {
  proposal <- current + rnorm(length(current), mean = 0, sd = proposal_sds)
  proposal_lp <- lp_fun(proposal, y, distance, buildingarea)

  log_alpha <- proposal_lp - current_lp
  accepted <- log(runif(1)) < log_alpha

  if (accepted) {
    list(state = proposal, lp = proposal_lp, accepted = TRUE)
  } else {
    list(state = current, lp = current_lp, accepted = FALSE)
  }
}

mh_mixture_with_lp <- function(y, distance, buildingarea, init, lp_fun,
                               n_iter = 30000,
                               proposal_sds_small = c(0.01, 0.001, 0.001, 0.01),
                               proposal_sds_large = c(0.03, 0.003, 0.003, 0.03),
                               mix_prob_small = 0.7) {
  samples <- matrix(NA_real_, nrow = n_iter, ncol = 4)
  colnames(samples) <- c("beta0", "beta1", "beta2", "eta")

  current <- init
  current_lp <- lp_fun(current, y, distance, buildingarea)
  accepted <- logical(n_iter)

  for (i in seq_len(n_iter)) {
    sds <- if (runif(1) < mix_prob_small) proposal_sds_small else proposal_sds_large
    out <- mh_step_with_lp(current, current_lp, sds, y, distance, buildingarea, lp_fun)
    current <- out$state
    current_lp <- out$lp
    accepted[i] <- out$accepted
    samples[i, ] <- current
  }

  list(samples = samples, acceptance_rate = mean(accepted))
}
```

Fitting

```
set.seed(123)

mh_draws_list <- vector("list", 4)
acc_rates <- numeric(4)

for (ch in 1:4) {
  out <- mh_mixture_with_lp(
    y = stan_data_model1$y,
    distance = stan_data_model1$distance,
    buildingarea = stan_data_model1$buildingarea,
    init = c(13.5, -0.1, 0.05, log(0.5)) + rnorm(4, 0, 0.05),
    lp_fun = log_posterior_model1,
    n_iter = 30000
  )

  draws <- as.data.frame(out$samples)
  draws$sigma <- exp(draws$eta)
  draws$iter <- 1:nrow(draws)
  draws$chain <- ch

  mh_draws_list[[ch]] <- draws
  acc_rates[ch] <- out$acceptance_rate
}

acc_rates
```

```
## [1] 0.3213000 0.3258333 0.3178333 0.3199000
```

```
mean(acc_rates)
```

```
## [1] 0.3212167
```

Burn-in for MH draws

```
burn_in_m1 <- 15000

mh_post_list <- lapply(mh_draws_list, function(df) {
  df %>% filter(iter > burn_in_m1)
})

mh_all_draws <- dplyr::bind_rows(mh_post_list)
head(mh_all_draws)
```

```
##      beta0       beta1      beta2        eta     sigma  iter chain
## 1 13.73938 -0.09744554 0.07457035 -0.6356742 0.5295783 15001     1
## 2 13.73938 -0.09744554 0.07457035 -0.6356742 0.5295783 15002     1
## 3 13.73938 -0.09744554 0.07457035 -0.6356742 0.5295783 15003     1
## 4 13.73938 -0.09744554 0.07457035 -0.6356742 0.5295783 15004     1
## 5 13.73938 -0.09744554 0.07457035 -0.6356742 0.5295783 15005     1
## 6 13.73938 -0.09744554 0.07457035 -0.6356742 0.5295783 15006     1
```

Posterior summaries for MH

```
library(dplyr)
library(tidyr)
library(tibble)
```

```
## Warning: package 'tibble' was built under R version 4.3.3
```

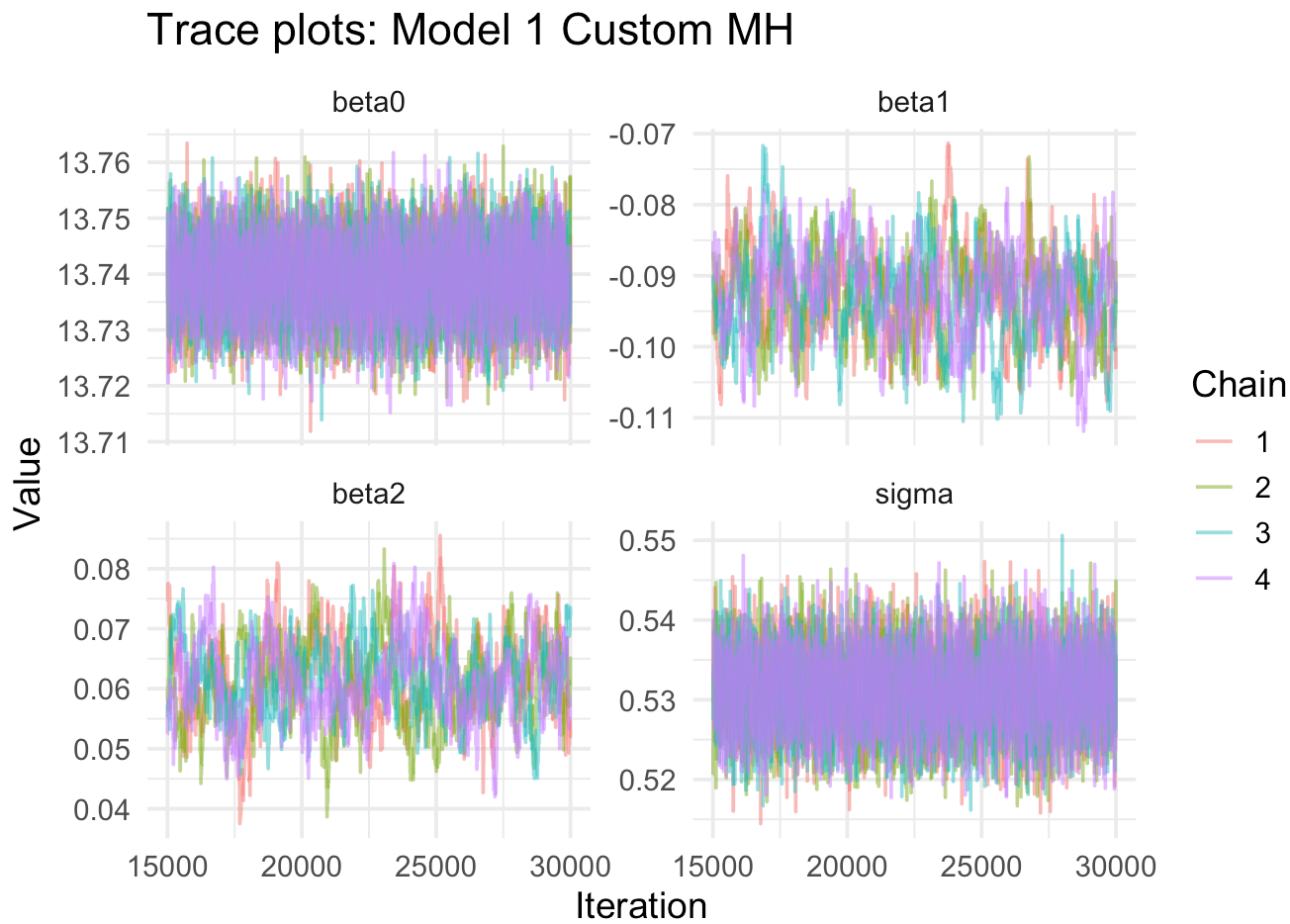
```r
library(posterior)

mh_m1_summary_basic <- mh_all_draws %>%
  summarise(
    beta0_mean   = mean(beta0),
    beta1_mean   = mean(beta1),
    beta2_mean   = mean(beta2),
    sigma_mean   = mean(sigma),

    beta0_median = median(beta0),
    beta1_median = median(beta1),
    beta2_median = median(beta2),
    sigma_median = median(sigma),

    beta0_sd     = sd(beta0),
    beta1_sd     = sd(beta1),
    beta2_sd     = sd(beta2),
    sigma_sd     = sd(sigma),

    beta0_q5     = quantile(beta0, 0.05),
    beta1_q5     = quantile(beta1, 0.05),
    beta2_q5     = quantile(beta2, 0.05),
    sigma_q5     = quantile(sigma, 0.05),

    beta0_q95    = quantile(beta0, 0.95),
    beta1_q95    = quantile(beta1, 0.95),
    beta2_q95    = quantile(beta2, 0.95),
    sigma_q95    = quantile(sigma, 0.95)
  ) %>%
  pivot_longer(
    cols = everything(),
    names_to = c("variable", ".value"),
    names_sep = "_"
  )

mh_vars <- c("beta0", "beta1", "beta2", "sigma")
n_chains <- length(mh_post_list)
n_iter_post <- nrow(mh_post_list[[1]])

mh_array <- array(
  NA_real_,
  dim = c(n_iter_post, n_chains, length(mh_vars)),
  dimnames = list(
    iteration = NULL,
    chain = paste0("chain", seq_len(n_chains)),
    variable = mh_vars
  )
)

for (ch in seq_len(n_chains)) {
  mh_array[, ch, ] <- as.matrix(mh_post_list[[ch]][, mh_vars])
}
```

```
mh_draws_array <- posterior::as_draws_array(mh_array)

mh_diag <- posterior::summarize_draws(
  mh_draws_array,
  rhat = posterior::rhat,
  ess_bulk = posterior::ess_bulk,
  ess_tail = posterior::ess_tail
)

if (".variable" %in% names(mh_diag)) {
  mh_diag <- mh_diag %>% rename(variable = .variable)
}

mh_diag <- mh_diag %>%
  select(variable, rhat, ess_bulk, ess_tail)

mh_m1_summary <- mh_m1_summary_basic %>%
  left_join(mh_diag, by = "variable") %>%
  select(variable, mean, median, sd, q5, q95, rhat, ess_bulk, ess_tail)

mh_m1_summary
```

```
## # A tibble: 4 × 9
##   variable    mean  median      sd      q5     q95  rhat ess_bulk ess_tail
##   <chr>      <dbl>   <dbl>   <dbl>   <dbl>   <dbl> <dbl>    <dbl>    <dbl>
## 1 beta0    13.7    13.7    0.00627 13.7    13.7     1.00    7701.    9436.
## 2 beta1    -0.0929 -0.0927 0.00603 -0.103  -0.0830  1.01     256.     511.
## 3 beta2     0.0613  0.0611 0.00653  0.0508  0.0723  1.01     181.     405.
## 4 sigma     0.531   0.531  0.00447  0.524   0.538   1.00    5027.    7576.
```

Trace plot

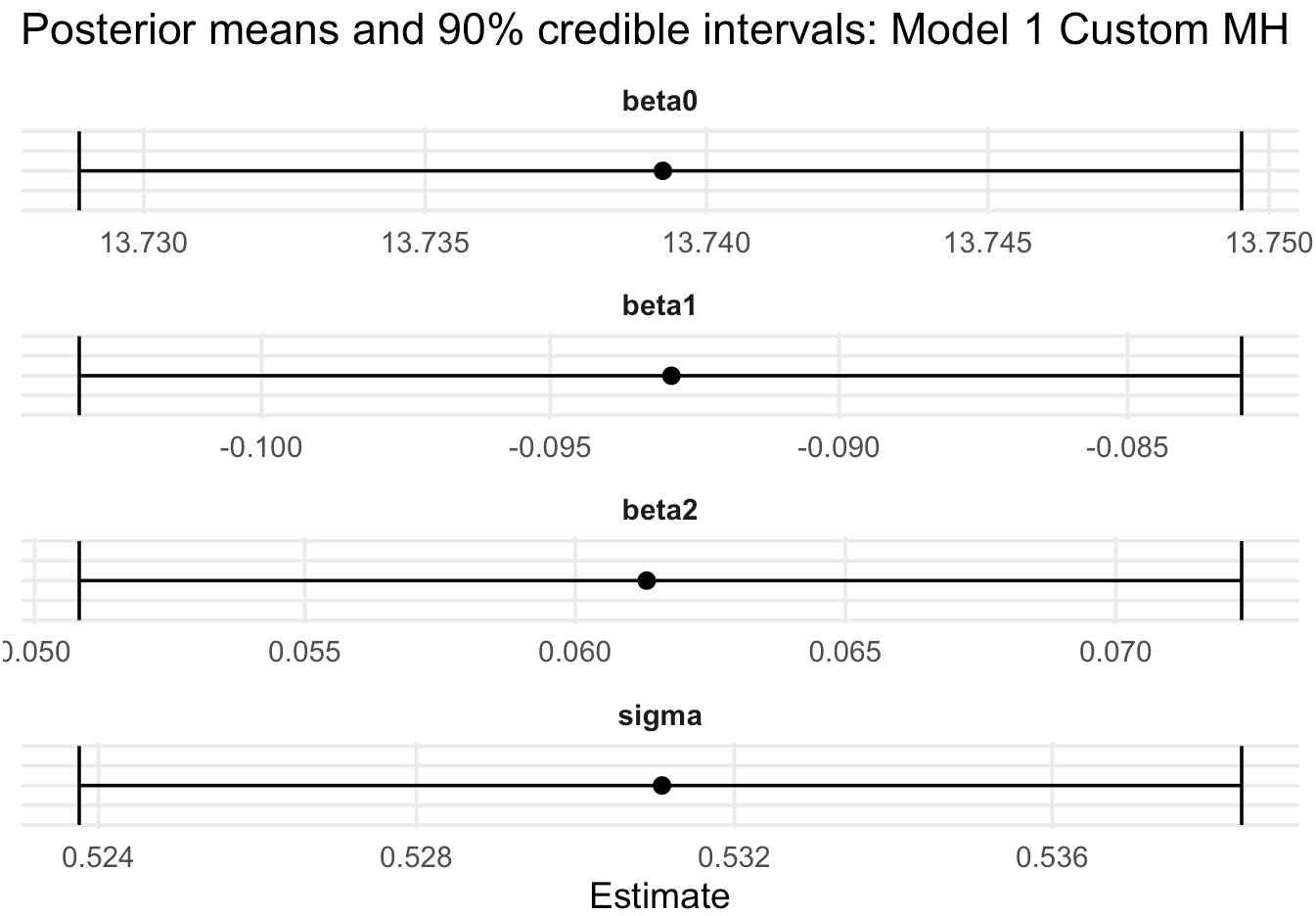
```
mh_all_draws %>%
  dplyr::select(iter, chain, beta0, beta1, beta2, sigma) %>%
  pivot_longer(cols = c(beta0, beta1, beta2, sigma), names_to = "parameter", values_to =
"value") %>%
  ggplot(aes(x = iter, y = value, colour = factor(chain))) +
  geom_line(alpha = 0.5) +
  facet_wrap(~ parameter, scales = "free_y", ncol = 2) +
  labs(title = "Trace plots: Model 1 Custom MH", x = "Iteration", y = "Value", colour =
"Chain") +
  theme_minimal(base_size = 14)
```



Credible Interval for model 1 custom MH

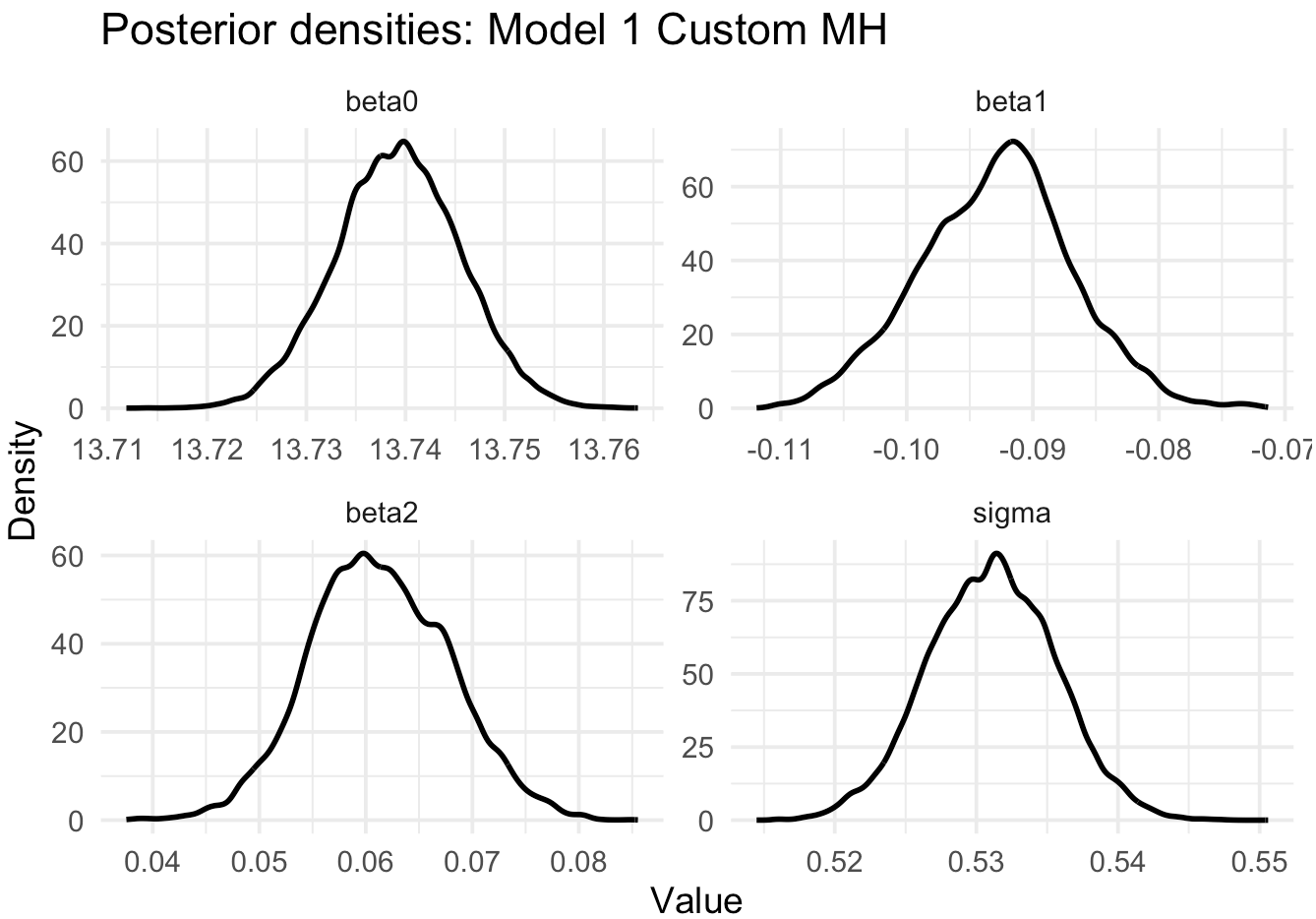
```
mh_m1_plot <- mh_m1_summary %>%
  filter(variable %in% c("beta0", "beta1", "beta2", "sigma"))

ggplot(mh_m1_plot, aes(x = mean, y = 1)) +
  geom_point(size = 2.5) +
  geom_errorbarh(aes(xmin = q5, xmax = q95), height = 0.08) +
  facet_wrap(~ variable, scales = "free_x", ncol = 1) +
  labs(
    title = "Posterior means and 90% credible intervals: Model 1 Custom MH",
    x = "Estimate",
    y = NULL
  ) +
  theme_minimal(base_size = 14) +
  theme(
    axis.text.y = element_blank(),
    axis.ticks.y = element_blank(),
    panel.grid.minor = element_blank(),
    strip.text = element_text(face = "bold")
  )
```

```
## `height` was translated to `width`.
```



Posterior densities for model 1 mh

```
mh_all_draws %>%
  dplyr::select(beta0, beta1, beta2, sigma) %>%
  pivot_longer(cols = everything(), names_to = "parameter", values_to = "value") %>%
  ggplot(aes(x = value)) +
  geom_density(linewidth = 1) +
  facet_wrap(~ parameter, scales = "free", ncol = 2) +
  labs(title = "Posterior densities: Model 1 Custom MH", x = "Value", y = "Density") +
  theme_minimal(base_size = 14)
```



MH convergence diagnostics

(used to compute for Gelman-Rubin and effective sample size)

```
mh_chain_list <- lapply(mh_post_list, function(df) {
  coda::mcmc(df[, c("beta0", "beta1", "beta2", "sigma")])
})

mh_mcmc_list <- coda::mcmc.list(mh_chain_list)

coda::gelman.diag(mh_mcmc_list, autoburnin = FALSE)
```

```
## Potential scale reduction factors:
##
##       Point est. Upper C.I.
## beta0       1.00       1.00
## beta1       1.00       1.01
## beta2       1.01       1.03
## sigma       1.00       1.00
##
## Multivariate psrf
##
## 1.01
```

```
coda::effectiveSize(mh_mcmc_list)
```

```
##     beta0     beta1     beta2     sigma
## 7408.6322  214.9354  196.2456 5395.8843
```

Invariance check

forward simulator

```
forward_model1_dataset <- function(part1_data) {
  beta0 <- rnorm(1, mean = 0, sd = 10)
  beta1 <- rnorm(1, mean = 0, sd = 1)
  beta2 <- rnorm(1, mean = 0, sd = 1)
  sigma <- rexp(1, rate = 1)

  mu <- beta0 + beta1 * part1_data$distance_z + beta2 * part1_data$buildingarea_z
  y_sim <- rnorm(nrow(part1_data), mean = mu, sd = sigma)

  list(
    theta = c(beta0 = beta0, beta1 = beta1, beta2 = beta2, eta = log(sigma)),
    data = list(
      y = y_sim,
      distance = part1_data$distance_z,
      buildingarea = part1_data$buildingarea_z
    )
  )
}
```

Test one MH chain from a forward draw

```
run_mh_from_forward_model1 <- function(part1_data,
                                       lp_fun,
                                       n_iter = 2000,
                                       proposal_sds_small = c(0.01, 0.001, 0.001, 0.01),
                                       proposal_sds_large = c(0.03, 0.003, 0.003, 0.03),
                                       mix_prob_small = 0.7) {
  sim <- forward_model1_dataset(part1_data)

  fit <- mh_mixture_with_lp(
    y = sim$data$y,
    distance = sim$data$distance,
    buildingarea = sim$data$buildingarea,
    init = sim$theta,
    lp_fun = lp_fun,
    n_iter = n_iter,
    proposal_sds_small = proposal_sds_small,
    proposal_sds_large = proposal_sds_large,
    mix_prob_small = mix_prob_small
  )

  list(
    initial_theta = sim$theta,
    final_theta = fit$samples[n_iter, ],
    acceptance_rate = fit$acceptance_rate
  )
}
```

```
run_invariance_test_model1 <- function(lp_fun,
                                       part1_data,
                                       n_rep = 500,
                                       n_iter_check = 2000,
                                       seed = 123,
                                       subset_size = 200,
                                       proposal_sds_small = c(0.01, 0.001, 0.001, 0.01),
                                       proposal_sds_large = c(0.03, 0.003, 0.003, 0.03),
                                       mix_prob_small = 0.7) {
  set.seed(seed)

  idx <- sample(seq_len(nrow(part1_data)),
                size = min(subset_size, nrow(part1_data)),
                replace = FALSE)

  part1_data_small <- part1_data[idx, , drop = FALSE]

  initial_mat <- matrix(NA_real_, nrow = n_rep, ncol = 4)
  final_mat   <- matrix(NA_real_, nrow = n_rep, ncol = 4)
  acc_rates   <- numeric(n_rep)

  colnames(initial_mat) <- c("beta0", "beta1", "beta2", "eta")
  colnames(final_mat)   <- c("beta0", "beta1", "beta2", "eta")

  for (m in seq_len(n_rep)) {
    out <- run_mh_from_forward_model1(
      part1_data = part1_data_small,
      lp_fun = lp_fun,
      n_iter = n_iter_check,
      proposal_sds_small = proposal_sds_small,
      proposal_sds_large = proposal_sds_large,
      mix_prob_small = mix_prob_small
    )

    initial_mat[m, ] <- out$initial_theta
    final_mat[m, ]   <- out$final_theta
    acc_rates[m]     <- out$acceptance_rate
  }

  initial_df <- as.data.frame(initial_mat)
  final_df   <- as.data.frame(final_mat)

  initial_df$sigma <- exp(initial_df$eta)
  final_df$sigma   <- exp(final_df$eta)

  ks_beta0 <- ks.test(jitter(initial_df$beta0, amount = 1e-10),
                      jitter(final_df$beta0, amount = 1e-10))

  ks_beta1 <- ks.test(jitter(initial_df$beta1, amount = 1e-10),
                      jitter(final_df$beta1, amount = 1e-10))

  ks_beta2 <- ks.test(jitter(initial_df$beta2, amount = 1e-10),
```

```
                      jitter(final_df$beta2, amount = 1e-10))

  ks_sigma <- ks.test(jitter(initial_df$sigma, amount = 1e-10),
                      jitter(final_df$sigma, amount = 1e-10))

  list(
    initial_df = initial_df,
    final_df = final_df,
    acceptance_rates = acc_rates,
    ks_beta0 = ks_beta0,
    ks_beta1 = ks_beta1,
    ks_beta2 = ks_beta2,
    ks_sigma = ks_sigma
  )
}
```

Clean invariance check

```
out_clean <- run_invariance_test_model1(
  lp_fun = log_posterior_model1,
  part1_data = part1_data,
  n_rep = 500,
  n_iter_check = 2000,
  seed = 123
)

out_clean$ks_beta0
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta0, amount = 1e-10) and jitter(final_df$beta0, amount = 1
e-10)
## D = 0.008, p-value = 1
## alternative hypothesis: two-sided
```

```
out_clean$ks_beta1
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta1, amount = 1e-10) and jitter(final_df$beta1, amount = 1
e-10)
## D = 0.012, p-value = 1
## alternative hypothesis: two-sided
```

```
out_clean$ks_beta2
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta2, amount = 1e-10) and jitter(final_df$beta2, amount = 1
e-10)
## D = 0.022, p-value = 0.9997
## alternative hypothesis: two-sided
```

```
out_clean$ks_sigma
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$sigma, amount = 1e-10) and jitter(final_df$sigma, amount = 1
e-10)
## D = 0.022, p-value = 0.9997
## alternative hypothesis: two-sided
```

Bug testing

B0 bug

```
log_posterior_model1_beta0_bug <- function(theta, y, distance, buildingarea) {
  beta0 <- theta[1]
  beta1 <- theta[2]
  beta2 <- theta[3]
  eta   <- theta[4]
  sigma <- exp(eta)

  mu <- beta0 + beta1 * distance + beta2 * buildingarea

  log_prior <-
    dnorm(beta0, mean = 20, sd = 0.25, log = TRUE) + # Bug is: wrong beta0 prior
    dnorm(beta1, mean = 0, sd = 1, log = TRUE) +
    dnorm(beta2, mean = 0, sd = 1, log = TRUE) +
    dexp(sigma, rate = 1, log = TRUE) + eta

  log_like <- sum(dnorm(y, mean = mu, sd = sigma, log = TRUE))

  log_prior + log_like
}
```

```
out_beta0_bug <- run_invariance_test_model1(
  lp_fun = log_posterior_model1_beta0_bug,
  part1_data = part1_data,
  n_rep = 500,
  n_iter_check = 2000,
  seed = 123
)

out_beta0_bug$ks_beta0
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta0, amount = 1e-10) and jitter(final_df$beta0, amount = 1
e-10)
## D = 0.334, p-value < 2.2e-16
## alternative hypothesis: two-sided
```

```
out_beta0_bug$ks_beta1
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta1, amount = 1e-10) and jitter(final_df$beta1, amount = 1
e-10)
## D = 0.018, p-value = 1
## alternative hypothesis: two-sided
```

```
out_beta0_bug$ks_beta2
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta2, amount = 1e-10) and jitter(final_df$beta2, amount = 1
e-10)
## D = 0.018, p-value = 1
## alternative hypothesis: two-sided
```

```
out_beta0_bug$ks_sigma
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$sigma, amount = 1e-10) and jitter(final_df$sigma, amount = 1
e-10)
## D = 0.646, p-value < 2.2e-16
## alternative hypothesis: two-sided
```

B1 bug

```
log_posterior_model1_beta1_bug <- function(theta, y, distance, buildingarea) {
  beta0 <- theta[1]
  beta1 <- theta[2]
  beta2 <- theta[3]
  eta   <- theta[4]
  sigma <- exp(eta)

  mu <- beta0 + beta1 * distance + beta2 * buildingarea

  log_prior <-
    dnorm(beta0, mean = 0, sd = 10, log = TRUE) +
    dnorm(beta1, mean = 0.03, sd = 0.005, log = TRUE) + # Bug is: wrong beta_1 prior
    dnorm(beta2, mean = 0, sd = 1, log = TRUE) +
    dexp(sigma, rate = 1, log = TRUE) + eta

  log_like <- sum(dnorm(y, mean = mu, sd = sigma, log = TRUE))

  log_prior + log_like
}
```

```
out_beta1_bug <- run_invariance_test_model1(
  lp_fun = log_posterior_model1_beta1_bug,
  part1_data = part1_data,
  n_rep = 500,
  n_iter_check = 2000,
  seed = 123
)

out_beta1_bug$ks_beta0
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta0, amount = 1e-10) and jitter(final_df$beta0, amount = 1
e-10)
## D = 0.01, p-value = 1
## alternative hypothesis: two-sided
```

```
out_beta1_bug$ks_beta1
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta1, amount = 1e-10) and jitter(final_df$beta1, amount = 1
e-10)
## D = 0.392, p-value < 2.2e-16
## alternative hypothesis: two-sided
```

```
out_beta1_bug$ks_beta2
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta2, amount = 1e-10) and jitter(final_df$beta2, amount = 1
e-10)
## D = 0.018, p-value = 1
## alternative hypothesis: two-sided
```

```
out_beta1_bug$ks_sigma
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$sigma, amount = 1e-10) and jitter(final_df$sigma, amount = 1
e-10)
## D = 0.29, p-value < 2.2e-16
## alternative hypothesis: two-sided
```

B2 bug

```
log_posterior_model1_beta2_bug <- function(theta, y, distance, buildingarea) {
  beta0 <- theta[1]
  beta1 <- theta[2]
  beta2 <- theta[3]
  eta   <- theta[4]
  sigma <- exp(eta)

  mu <- beta0 + beta1 * distance + beta2 * 10 * buildingarea  # Bug is: beta2 effect is
multiplied by 10 in the likelihood mean

  log_prior <-
    dnorm(beta0, mean = 0, sd = 10, log = TRUE) +
    dnorm(beta1, mean = 0, sd = 1, log = TRUE) +
    dnorm(beta2, mean = 0, sd = 1, log = TRUE) +
    dexp(sigma, rate = 1, log = TRUE) + eta

  log_like <- sum(dnorm(y, mean = mu, sd = sigma, log = TRUE))

  log_prior + log_like
}
```

```
out_beta2_bug <- run_invariance_test_model1(
  lp_fun = log_posterior_model1_beta2_bug,
  part1_data = part1_data,
  n_rep = 500,
  n_iter_check = 2000,
  seed = 123
)

out_beta2_bug$ks_beta0
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta0, amount = 1e-10) and jitter(final_df$beta0, amount = 1
e-10)
## D = 0.01, p-value = 1
## alternative hypothesis: two-sided
```

```
out_beta2_bug$ks_beta1
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta1, amount = 1e-10) and jitter(final_df$beta1, amount = 1
e-10)
## D = 0.018, p-value = 1
## alternative hypothesis: two-sided
```

```
out_beta2_bug$ks_beta2
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta2, amount = 1e-10) and jitter(final_df$beta2, amount = 1
e-10)
## D = 0.14, p-value = 0.0001109
## alternative hypothesis: two-sided
```

```
out_beta2_bug$ks_sigma
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$sigma, amount = 1e-10) and jitter(final_df$sigma, amount = 1
e-10)
## D = 0.194, p-value = 1.344e-08
## alternative hypothesis: two-sided
```

Sigma fail

```
log_posterior_model1_sigma_bug <- function(theta, y, distance, buildingarea) {
  beta0 <- theta[1]
  beta1 <- theta[2]
  beta2 <- theta[3]
  eta   <- theta[4]
  sigma <- exp(eta)

  mu <- beta0 + beta1 * distance + beta2 * buildingarea

  log_prior <-
    dnorm(beta0, mean = 0, sd = 10, log = TRUE) +
    dnorm(beta1, mean = 0, sd = 1, log = TRUE) +
    dnorm(beta2, mean = 0, sd = 1, log = TRUE) +
    dexp(sigma, rate = 1, log = TRUE) + eta

  log_like <- sum(dnorm(y, mean = mu, sd = sigma / 4, log = TRUE))  # Bug is the wrong s
igma used in likelihood scale

  log_prior + log_like
}
```

```
out_sigma_bug <- run_invariance_test_model1(
  lp_fun = log_posterior_model1_sigma_bug,
  part1_data = part1_data,
  n_rep = 500,
  n_iter_check = 2000,
  seed = 123
)

out_beta2_bug$ks_beta0
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta0, amount = 1e-10) and jitter(final_df$beta0, amount = 1
e-10)
## D = 0.01, p-value = 1
## alternative hypothesis: two-sided
```

```
out_beta2_bug$ks_beta1
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta1, amount = 1e-10) and jitter(final_df$beta1, amount = 1
e-10)
## D = 0.018, p-value = 1
## alternative hypothesis: two-sided
```

```
out_beta2_bug$ks_beta2
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$beta2, amount = 1e-10) and jitter(final_df$beta2, amount = 1
e-10)
## D = 0.14, p-value = 0.0001109
## alternative hypothesis: two-sided
```

```
out_beta2_bug$ks_sigma
```

```
##
##  Asymptotic two-sample Kolmogorov-Smirnov test
##
## data:  jitter(initial_df$sigma, amount = 1e-10) and jitter(final_df$sigma, amount = 1
e-10)
## D = 0.194, p-value = 1.344e-08
## alternative hypothesis: two-sided
```

Stan vs Custom MH for model 1

summary comparison table (won’t show in doc)

```
stan_m1_summary <- fit_m1$summary(variables = c("beta0", "beta1", "beta2", "sigma")) %>%
  as.data.frame() %>%
  dplyr::select(variable, mean, median, sd, q5, q95) %>%
  mutate(method = "Stan")

mh_m1_summary2 <- mh_all_draws %>%
  summarise(
    beta0_mean = mean(beta0), beta1_mean = mean(beta1), beta2_mean = mean(beta2), sigma_
mean = mean(sigma),
    beta0_median = median(beta0), beta1_median = median(beta1), beta2_median = median(be
ta2), sigma_median = median(sigma),
    beta0_sd = sd(beta0), beta1_sd = sd(beta1), beta2_sd = sd(beta2), sigma_sd = sd(sigm
a),
    beta0_q5 = quantile(beta0, 0.05), beta1_q5 = quantile(beta1, 0.05), beta2_q5 = quant
ile(beta2, 0.05), sigma_q5 = quantile(sigma, 0.05),
    beta0_q95 = quantile(beta0, 0.95), beta1_q95 = quantile(beta1, 0.95), beta2_q95 = qu
antile(beta2, 0.95), sigma_q95 = quantile(sigma, 0.95)
  ) %>%
  pivot_longer(cols = everything(), names_to = c("variable", ".value"), names_sep = "_")
%>%
  mutate(method = "Custom MH")

comparison_table_m1 <- dplyr::bind_rows(stan_m1_summary, mh_m1_summary2) %>%
  arrange(variable, method)

comparison_table_m1
```

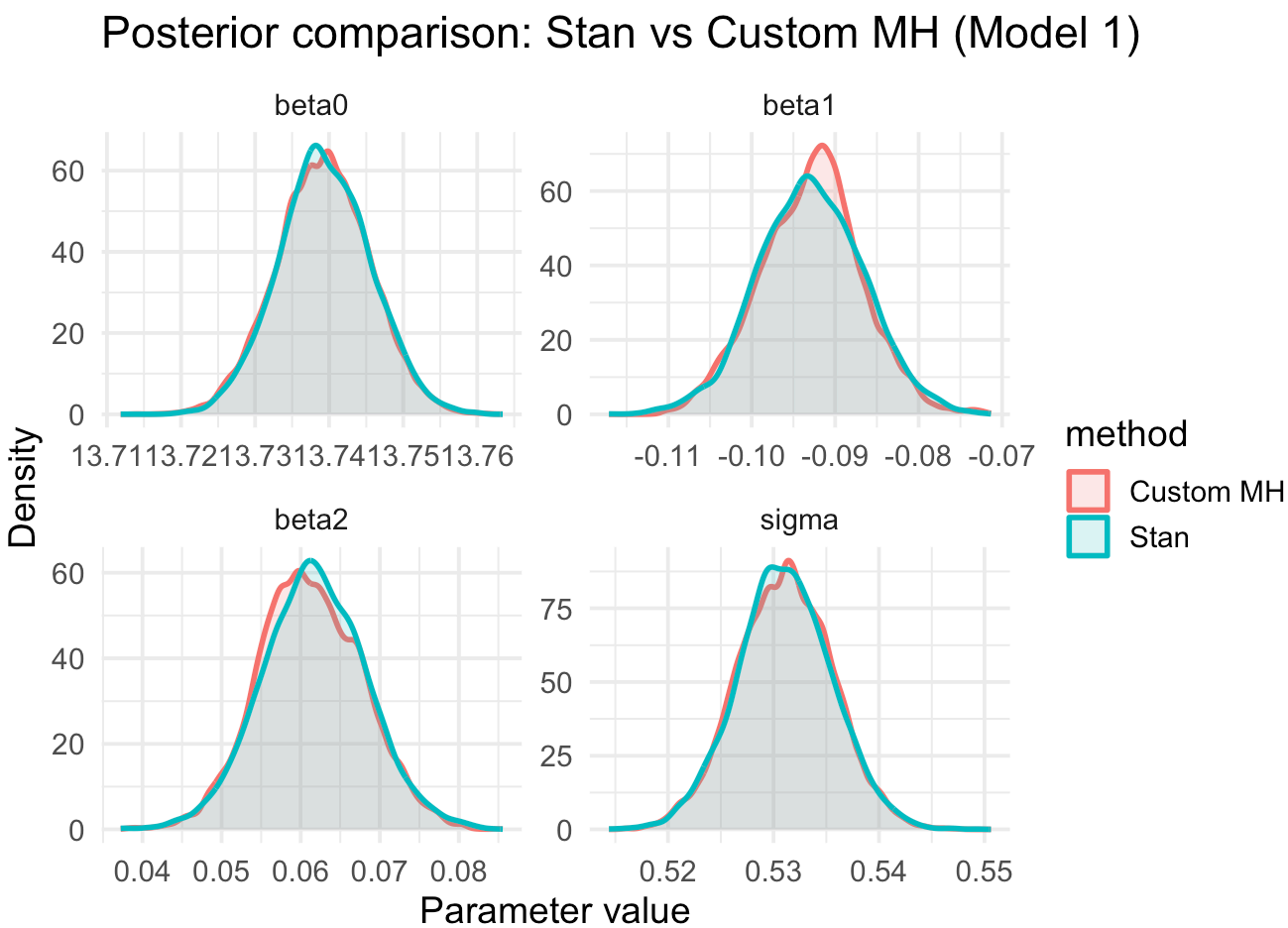
```
##   variable        mean      median          sd          q5         q95
## 1    beta0 13.73922490 13.73929124 0.006268365 13.72885487 13.74950993
## 2    beta0 13.73941273 13.73921300 0.006139360 13.72925830 13.74960830
## 3    beta1 -0.09290203 -0.09266517 0.006033493 -0.10315694 -0.08302731
## 4    beta1 -0.09291254 -0.09300522 0.006276566 -0.10294222 -0.08262237
## 5    beta2  0.06132163  0.06107597 0.006530228  0.05082790  0.07232580
## 6    beta2  0.06173576  0.06167671 0.006555387  0.05105858  0.07252791
## 7    sigma  0.53109012  0.53113184 0.004474404  0.52375775  0.53838352
## 8    sigma  0.53106841  0.53102125 0.004452695  0.52371207  0.53844117
##      method
## 1 Custom MH
## 2      Stan
## 3 Custom MH
## 4      Stan
## 5 Custom MH
## 6      Stan
## 7 Custom MH
## 8      Stan
```

Method comparison check with density overlay

```
plot_draws_m1 <- dplyr::bind_rows(
  mh_all_draws %>% dplyr::select(beta0, beta1, beta2, sigma) %>% mutate(method = "Custom
MH"),
  fit_m1$draws(variables = c("beta0", "beta1", "beta2", "sigma"), format = "df") %>%
    dplyr::select(beta0, beta1, beta2, sigma) %>%
    mutate(method = "Stan")
) %>%
  pivot_longer(cols = c(beta0, beta1, beta2, sigma), names_to = "parameter", values_to =
"value")
```

```
## Warning: Dropping 'draws_df' class as required metadata was removed.
```

```
ggplot(plot_draws_m1, aes(x = value, colour = method, fill = method)) +
  geom_density(alpha = 0.18, linewidth = 1) +
  facet_wrap(~ parameter, scales = "free", ncol = 2) +
  labs(title = "Posterior comparison: Stan vs Custom MH (Model 1)", x = "Parameter valu
e", y = "Density") +
  theme_minimal(base_size = 14)
```



Model 2 with stan

```
mod2 <- cmdstan_model("Part2_Model2_Hierarchical.stan")
fit_m2 <- mod2$sample(
  data = stan_data_model2,
  chains = 4,
  parallel_chains = 4,
  iter_warmup = 500,
  iter_sampling = 1000,
  seed = 123,
  init = 0,
  refresh = 0,
  adapt_delta = 0.99,
  max_treedepth = 12
)
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 3 finished in 139.7 seconds.
## Chain 1 finished in 205.9 seconds.
## Chain 4 finished in 212.3 seconds.
## Chain 2 finished in 278.8 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 209.2 seconds.
## Total execution time: 278.9 seconds.
```

Posterior summaries

```
summ_m2 <- fit_m2$summary(variables = c("beta0", "beta1", "beta2", "tau", "sigma")) %>%
  as.data.frame() %>%
  dplyr::select(variable, mean, median, sd, q5, q95, rhat, ess_bulk, ess_tail)

summ_m2
```

```
##   variable       mean     median          sd          q5         q95     rhat
## 1    beta0 13.7235344 13.7202835 0.102107431 13.55925320 13.89138930 1.004792
## 2    beta1 -0.1200635 -0.1199646 0.007446672 -0.13260026 -0.10782002 1.000433
## 3    beta2  0.0615728  0.0615062 0.005942511  0.05206021  0.07148747 1.000171
## 4      tau  0.2739792  0.2535603 0.094000185  0.16371021  0.45432845 1.005759
## 5    sigma  0.4946015  0.4944574 0.004155128  0.48774664  0.50131679 1.003679
##    ess_bulk ess_tail
## 1  797.4561 1191.270
## 2 3627.4467 2960.077
## 3 3832.5763 2418.467
## 4  790.2834 1709.374
## 5 3127.8076 2656.871
```

Sampler diagnostics

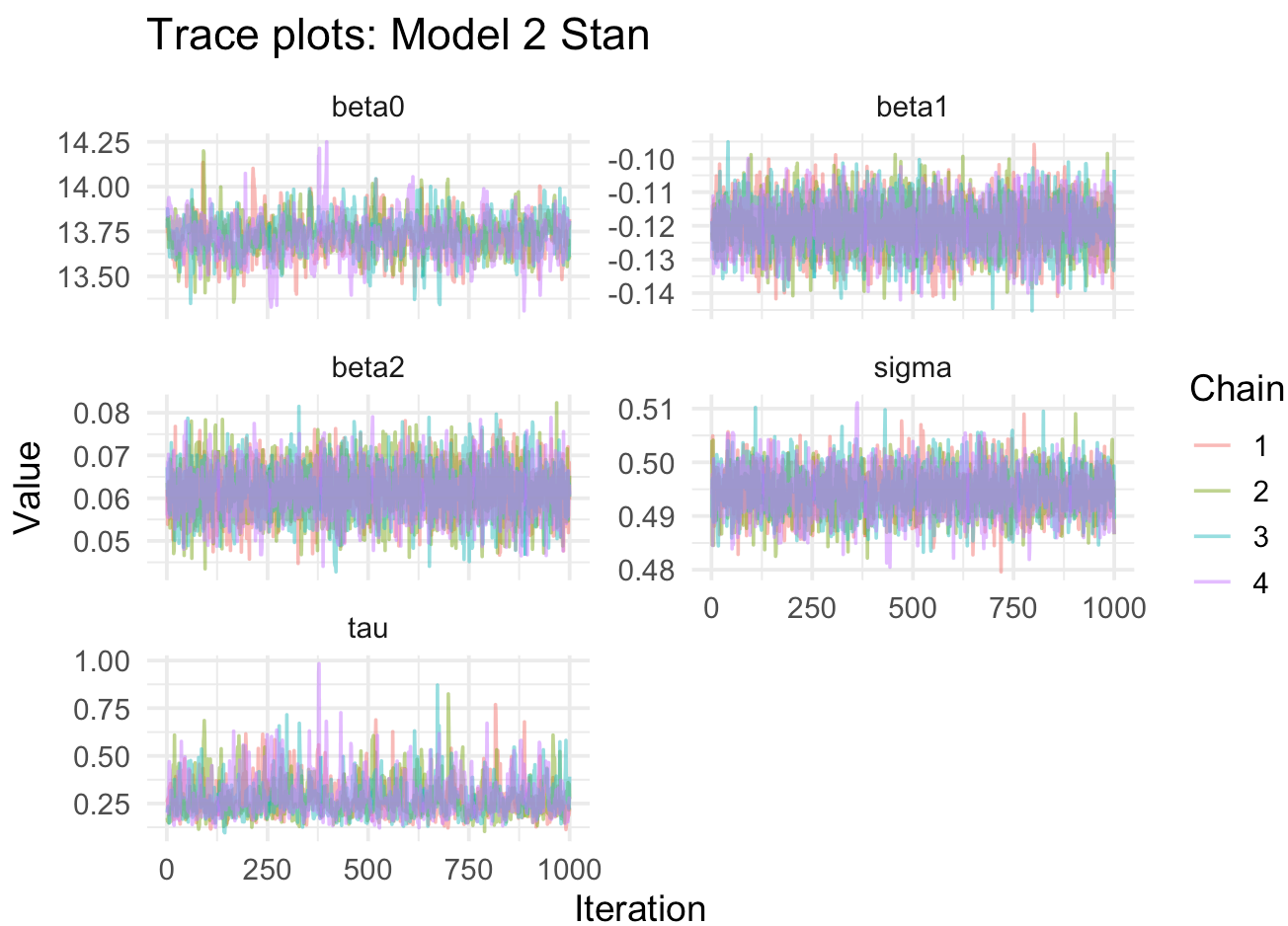
```
check_sampler_issues(fit_m2, 12)
```

```
##   divergent hit_max_treedepth max_treedepth_seen
## 1         0                 0                  9
```

Trace plot

```
fit_m2$draws(variables = c("beta0", "beta1", "beta2", "tau", "sigma"), format = "draws_d
f") %>%
  dplyr::select(.chain, .iteration, beta0, beta1, beta2, tau, sigma) %>%
  pivot_longer(cols = c(beta0, beta1, beta2, tau, sigma), names_to = "parameter", values
_to = "value") %>%
  ggplot(aes(x = .iteration, y = value, colour = factor(.chain))) +
  geom_line(alpha = 0.5) +
  facet_wrap(~ parameter, scales = "free_y", ncol = 2) +
  labs(title = "Trace plots: Model 2 Stan", x = "Iteration", y = "Value", colour = "Chai
n") +
  theme_minimal(base_size = 14)
```

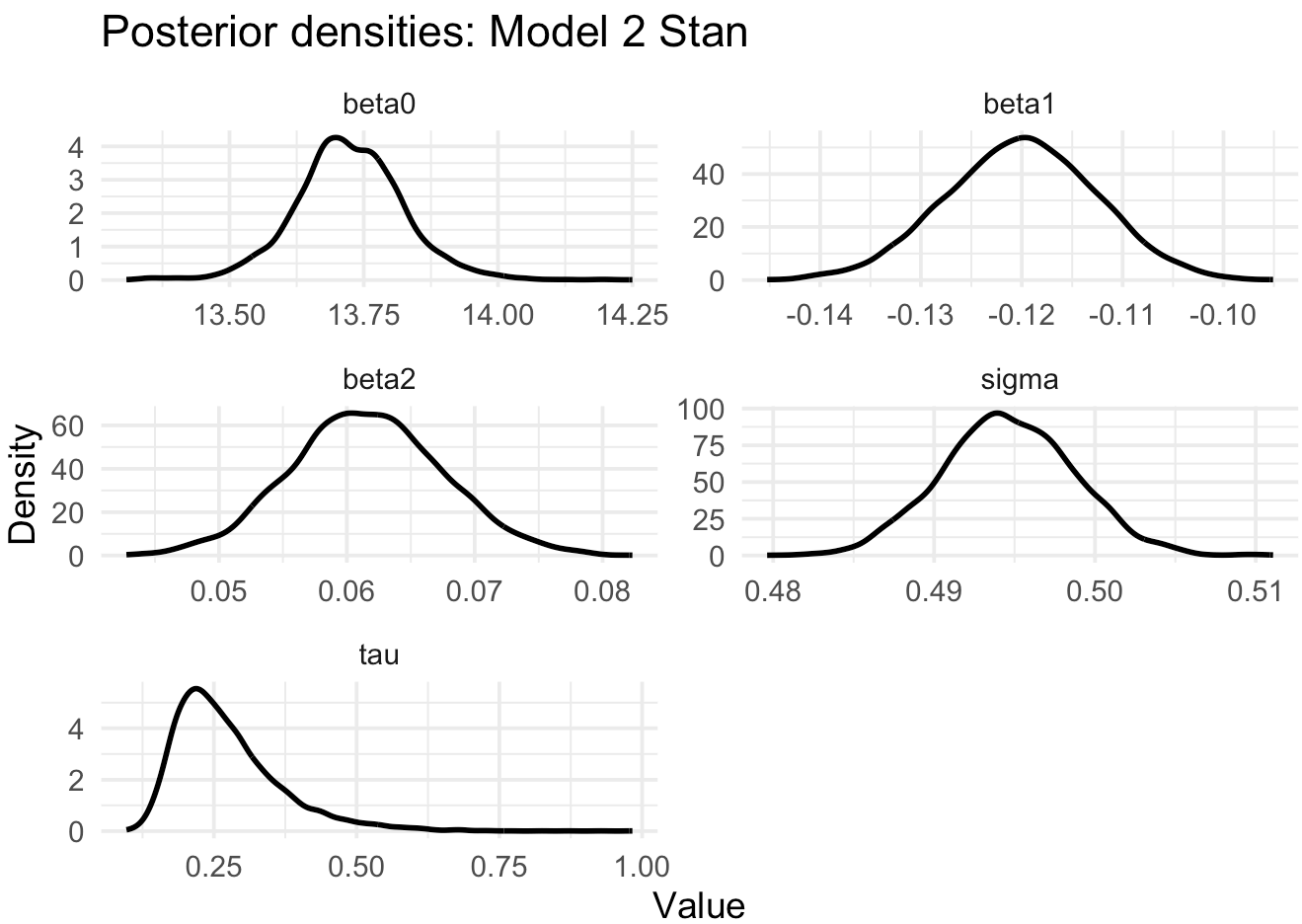
```
## Warning: Dropping 'draws_df' class as required metadata was removed.
```



Posterior densities

```
fit_m2$draws(variables = c("beta0", "beta1", "beta2", "tau", "sigma"), format = "draws_d
f") %>%
  dplyr::select(beta0, beta1, beta2, tau, sigma) %>%
  pivot_longer(cols = everything(), names_to = "parameter", values_to = "value") %>%
  ggplot(aes(x = value)) +
  geom_density(linewidth = 1) +
  facet_wrap(~ parameter, scales = "free", ncol = 2) +
  labs(title = "Posterior densities: Model 2 Stan", x = "Value", y = "Density") +
  theme_minimal(base_size = 14)
```

```
## Warning: Dropping 'draws_df' class as required metadata was removed.
```



MCMC Convergence diagnostics

```
m2_draws <- fit_m2$draws(
  variables = c("beta0", "beta1", "beta2", "tau", "sigma"),
  format = "draws_df"
) %>%
  as.data.frame() %>%
  dplyr::select(.chain, beta0, beta1, beta2, tau, sigma)
```

```
m2_chain_list <- split(m2_draws, m2_draws$.chain)

m2_chain_list <- lapply(m2_chain_list, function(df) {
  coda::mcmc(df[, c("beta0", "beta1", "beta2", "tau", "sigma")])
})

m2_mcmc_list <- coda::mcmc.list(m2_chain_list)
```

```
coda::gelman.diag(m2_mcmc_list, autoburnin = FALSE)
```

```
## Potential scale reduction factors:
##
##       Point est. Upper C.I.
## beta0       1.01       1.02
## beta1       1.00       1.00
## beta2       1.00       1.00
## tau         1.01       1.02
## sigma       1.00       1.00
##
## Multivariate psrf
##
## 1.01
```

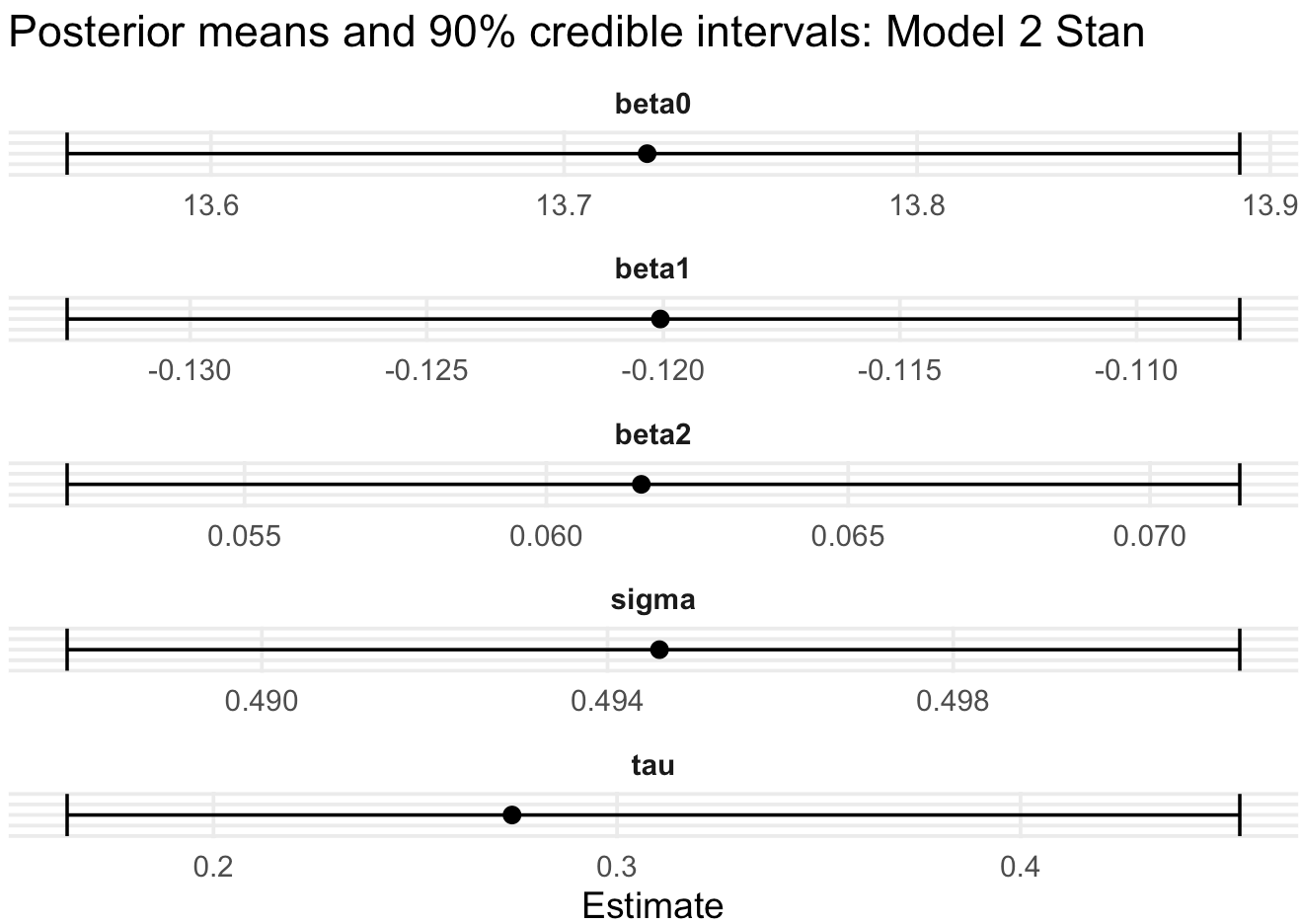
```
coda::effectiveSize(m2_mcmc_list)
```

```
##     beta0     beta1     beta2       tau     sigma
##  882.3492 3700.6374 3876.2570  934.5770 3463.0424
```

Credible intervals for model 2 stan

```
summm_m2_plot <- summm_m2 %>%
  filter(variable %in% c("beta0", "beta1", "beta2", "tau", "sigma"))

ggplot(summm_m2_plot, aes(x = mean, y = 1)) +
  geom_point(size = 2.5) +
  geom_errorbarh(aes(xmin = q5, xmax = q95), height = 0.08) +
  facet_wrap(~ variable, scales = "free_x", ncol = 1) +
  labs(
    title = "Posterior means and 90% credible intervals: Model 2 Stan",
    x = "Estimate",
    y = NULL
  ) +
  theme_minimal(base_size = 14) +
  theme(
    axis.text.y = element_blank(),
    axis.ticks.y = element_blank(),
    panel.grid.minor = element_blank(),
    strip.text = element_text(face = "bold")
  )
```

```
## `height` was translated to `width`.
```



Prior Sensitivity check - robustness check on prior choices

fitting

```
mod2_sens <- cmdstan_model("model2_sensitivity.stan")
fits_m2_sens <- list()
prior_settings_m2 <- list(
  baseline = list(beta0_sd = 10, beta_sd = 1, tau_rate = 1, sigma_rate = 1),
  diffuse  = list(beta0_sd = 20, beta_sd = 2, tau_rate = 0.5, sigma_rate = 0.5),
  regular  = list(beta0_sd = 5, beta_sd = 0.5, tau_rate = 2, sigma_rate = 2)
)

for (nm in names(prior_settings_m2)) {
  cat("\nRunning prior setting:", nm, "\n")

  pr <- prior_settings_m2[[nm]]

  fits_m2_sens[[nm]] <- mod2_sens$sample(
    data = c(
      list(
        N = stan_data_model2$N,
        R = stan_data_model2$R,
        y = stan_data_model2$y,
        distance = stan_data_model2$distance,
        buildingarea = stan_data_model2$buildingarea,
        region = stan_data_model2$region
      ),
      pr
    ),
    chains = 4,
    parallel_chains = 4,
    iter_warmup = 500,
    iter_sampling = 1000,
    seed = 123,
    init = 0,
    refresh = 100,
    adapt_delta = 0.95,
    max_treedepth = 12
  )
}
```

```
##
## Running prior setting: baseline
## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 1500 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 1500 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 1500 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 1500 [  0%]  (Warmup)
## Chain 2 Iteration:  100 / 1500 [  6%]  (Warmup)
## Chain 4 Iteration:  100 / 1500 [  6%]  (Warmup)
## Chain 1 Iteration:  100 / 1500 [  6%]  (Warmup)
## Chain 3 Iteration:  100 / 1500 [  6%]  (Warmup)
## Chain 3 Iteration:  200 / 1500 [ 13%]  (Warmup)
## Chain 3 Iteration:  300 / 1500 [ 20%]  (Warmup)
## Chain 3 Iteration:  400 / 1500 [ 26%]  (Warmup)
## Chain 3 Iteration:  500 / 1500 [ 33%]  (Warmup)
## Chain 3 Iteration:  501 / 1500 [ 33%]  (Sampling)
## Chain 3 Iteration:  600 / 1500 [ 40%]  (Sampling)
## Chain 4 Iteration:  200 / 1500 [ 13%]  (Warmup)
## Chain 3 Iteration:  700 / 1500 [ 46%]  (Sampling)
## Chain 3 Iteration:  800 / 1500 [ 53%]  (Sampling)
## Chain 2 Iteration:  200 / 1500 [ 13%]  (Warmup)
## Chain 3 Iteration:  900 / 1500 [ 60%]  (Sampling)
## Chain 3 Iteration: 1000 / 1500 [ 66%]  (Sampling)
## Chain 3 Iteration: 1100 / 1500 [ 73%]  (Sampling)
## Chain 1 Iteration:  200 / 1500 [ 13%]  (Warmup)
## Chain 4 Iteration:  300 / 1500 [ 20%]  (Warmup)
## Chain 3 Iteration: 1200 / 1500 [ 80%]  (Sampling)
## Chain 4 Iteration:  400 / 1500 [ 26%]  (Warmup)
## Chain 3 Iteration: 1300 / 1500 [ 86%]  (Sampling)
## Chain 4 Iteration:  500 / 1500 [ 33%]  (Warmup)
## Chain 4 Iteration:  501 / 1500 [ 33%]  (Sampling)
## Chain 3 Iteration: 1400 / 1500 [ 93%]  (Sampling)
## Chain 4 Iteration:  600 / 1500 [ 40%]  (Sampling)
## Chain 3 Iteration: 1500 / 1500 [100%]  (Sampling)
## Chain 3 finished in 69.7 seconds.
## Chain 4 Iteration:  700 / 1500 [ 46%]  (Sampling)
## Chain 1 Iteration:  300 / 1500 [ 20%]  (Warmup)
## Chain 4 Iteration:  800 / 1500 [ 53%]  (Sampling)
## Chain 1 Iteration:  400 / 1500 [ 26%]  (Warmup)
## Chain 4 Iteration:  900 / 1500 [ 60%]  (Sampling)
## Chain 1 Iteration:  500 / 1500 [ 33%]  (Warmup)
## Chain 1 Iteration:  501 / 1500 [ 33%]  (Sampling)
## Chain 2 Iteration:  300 / 1500 [ 20%]  (Warmup)
## Chain 4 Iteration: 1000 / 1500 [ 66%]  (Sampling)
## Chain 1 Iteration:  600 / 1500 [ 40%]  (Sampling)
## Chain 4 Iteration: 1100 / 1500 [ 73%]  (Sampling)
## Chain 4 Iteration: 1200 / 1500 [ 80%]  (Sampling)
## Chain 1 Iteration:  700 / 1500 [ 46%]  (Sampling)
## Chain 4 Iteration: 1300 / 1500 [ 86%]  (Sampling)
## Chain 2 Iteration:  400 / 1500 [ 26%]  (Warmup)
## Chain 1 Iteration:  800 / 1500 [ 53%]  (Sampling)
```

```
## Chain 4 Iteration: 1400 / 1500 [ 93%]  (Sampling)
## Chain 4 Iteration: 1500 / 1500 [100%]  (Sampling)
## Chain 4 finished in 104.8 seconds.
## Chain 1 Iteration:  900 / 1500 [ 60%]  (Sampling)
## Chain 2 Iteration:  500 / 1500 [ 33%]  (Warmup)
## Chain 2 Iteration:  501 / 1500 [ 33%]  (Sampling)
## Chain 1 Iteration: 1000 / 1500 [ 66%]  (Sampling)
## Chain 2 Iteration:  600 / 1500 [ 40%]  (Sampling)
## Chain 1 Iteration: 1100 / 1500 [ 73%]  (Sampling)
## Chain 2 Iteration:  700 / 1500 [ 46%]  (Sampling)
## Chain 2 Iteration:  800 / 1500 [ 53%]  (Sampling)
## Chain 1 Iteration: 1200 / 1500 [ 80%]  (Sampling)
## Chain 2 Iteration:  900 / 1500 [ 60%]  (Sampling)
## Chain 1 Iteration: 1300 / 1500 [ 86%]  (Sampling)
## Chain 2 Iteration: 1000 / 1500 [ 66%]  (Sampling)
## Chain 2 Iteration: 1100 / 1500 [ 73%]  (Sampling)
## Chain 1 Iteration: 1400 / 1500 [ 93%]  (Sampling)
## Chain 2 Iteration: 1200 / 1500 [ 80%]  (Sampling)
## Chain 1 Iteration: 1500 / 1500 [100%]  (Sampling)
## Chain 1 finished in 139.2 seconds.
## Chain 2 Iteration: 1300 / 1500 [ 86%]  (Sampling)
## Chain 2 Iteration: 1400 / 1500 [ 93%]  (Sampling)
## Chain 2 Iteration: 1500 / 1500 [100%]  (Sampling)
## Chain 2 finished in 149.2 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 115.7 seconds.
## Total execution time: 149.3 seconds.
##
##
## Running prior setting: diffuse
## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 1500 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 1500 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 1500 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 1500 [  0%]  (Warmup)
## Chain 4 Iteration:  100 / 1500 [  6%]  (Warmup)
## Chain 3 Iteration:  100 / 1500 [  6%]  (Warmup)
## Chain 2 Iteration:  100 / 1500 [  6%]  (Warmup)
## Chain 1 Iteration:  100 / 1500 [  6%]  (Warmup)
## Chain 1 Iteration:  200 / 1500 [ 13%]  (Warmup)
## Chain 3 Iteration:  200 / 1500 [ 13%]  (Warmup)
## Chain 1 Iteration:  300 / 1500 [ 20%]  (Warmup)
## Chain 3 Iteration:  300 / 1500 [ 20%]  (Warmup)
## Chain 1 Iteration:  400 / 1500 [ 26%]  (Warmup)
## Chain 3 Iteration:  400 / 1500 [ 26%]  (Warmup)
## Chain 1 Iteration:  500 / 1500 [ 33%]  (Warmup)
## Chain 1 Iteration:  501 / 1500 [ 33%]  (Sampling)
## Chain 1 Iteration:  600 / 1500 [ 40%]  (Sampling)
## Chain 3 Iteration:  500 / 1500 [ 33%]  (Warmup)
## Chain 3 Iteration:  501 / 1500 [ 33%]  (Sampling)
```

```
## Chain 1 Iteration:  700 / 1500 [ 46%]  (Sampling)
## Chain 3 Iteration:  600 / 1500 [ 40%]  (Sampling)
## Chain 1 Iteration:  800 / 1500 [ 53%]  (Sampling)
## Chain 3 Iteration:  700 / 1500 [ 46%]  (Sampling)
## Chain 2 Iteration:  200 / 1500 [ 13%]  (Warmup)
## Chain 1 Iteration:  900 / 1500 [ 60%]  (Sampling)
## Chain 1 Iteration: 1000 / 1500 [ 66%]  (Sampling)
## Chain 2 Iteration:  300 / 1500 [ 20%]  (Warmup)
## Chain 3 Iteration:  800 / 1500 [ 53%]  (Sampling)
## Chain 4 Iteration:  200 / 1500 [ 13%]  (Warmup)
## Chain 2 Iteration:  400 / 1500 [ 26%]  (Warmup)
## Chain 1 Iteration: 1100 / 1500 [ 73%]  (Sampling)
## Chain 3 Iteration:  900 / 1500 [ 60%]  (Sampling)
## Chain 2 Iteration:  500 / 1500 [ 33%]  (Warmup)
## Chain 2 Iteration:  501 / 1500 [ 33%]  (Sampling)
## Chain 1 Iteration: 1200 / 1500 [ 80%]  (Sampling)
## Chain 2 Iteration:  600 / 1500 [ 40%]  (Sampling)
## Chain 3 Iteration: 1000 / 1500 [ 66%]  (Sampling)
## Chain 1 Iteration: 1300 / 1500 [ 86%]  (Sampling)
## Chain 2 Iteration:  700 / 1500 [ 46%]  (Sampling)
## Chain 1 Iteration: 1400 / 1500 [ 93%]  (Sampling)
## Chain 3 Iteration: 1100 / 1500 [ 73%]  (Sampling)
## Chain 2 Iteration:  800 / 1500 [ 53%]  (Sampling)
## Chain 1 Iteration: 1500 / 1500 [100%]  (Sampling)
## Chain 1 finished in 87.9 seconds.
## Chain 3 Iteration: 1200 / 1500 [ 80%]  (Sampling)
## Chain 2 Iteration:  900 / 1500 [ 60%]  (Sampling)
## Chain 3 Iteration: 1300 / 1500 [ 86%]  (Sampling)
## Chain 2 Iteration: 1000 / 1500 [ 66%]  (Sampling)
## Chain 3 Iteration: 1400 / 1500 [ 93%]  (Sampling)
## Chain 2 Iteration: 1100 / 1500 [ 73%]  (Sampling)
## Chain 2 Iteration: 1200 / 1500 [ 80%]  (Sampling)
## Chain 3 Iteration: 1500 / 1500 [100%]  (Sampling)
## Chain 3 finished in 105.5 seconds.
## Chain 2 Iteration: 1300 / 1500 [ 86%]  (Sampling)
## Chain 2 Iteration: 1400 / 1500 [ 93%]  (Sampling)
## Chain 2 Iteration: 1500 / 1500 [100%]  (Sampling)
## Chain 2 finished in 116.9 seconds.
## Chain 4 Iteration:  300 / 1500 [ 20%]  (Warmup)
## Chain 4 Iteration:  400 / 1500 [ 26%]  (Warmup)
## Chain 4 Iteration:  500 / 1500 [ 33%]  (Warmup)
## Chain 4 Iteration:  501 / 1500 [ 33%]  (Sampling)
## Chain 4 Iteration:  600 / 1500 [ 40%]  (Sampling)
## Chain 4 Iteration:  700 / 1500 [ 46%]  (Sampling)
## Chain 4 Iteration:  800 / 1500 [ 53%]  (Sampling)
## Chain 4 Iteration:  900 / 1500 [ 60%]  (Sampling)
## Chain 4 Iteration: 1000 / 1500 [ 66%]  (Sampling)
## Chain 4 Iteration: 1100 / 1500 [ 73%]  (Sampling)
## Chain 4 Iteration: 1200 / 1500 [ 80%]  (Sampling)
## Chain 4 Iteration: 1300 / 1500 [ 86%]  (Sampling)
## Chain 4 Iteration: 1400 / 1500 [ 93%]  (Sampling)
## Chain 4 Iteration: 1500 / 1500 [100%]  (Sampling)
```

```
## Chain 4 finished in 177.0 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 121.8 seconds.
## Total execution time: 177.1 seconds.
##
##
## Running prior setting: regular
## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 1500 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 1500 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 1500 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 1500 [  0%]  (Warmup)
## Chain 2 Iteration:  100 / 1500 [  6%]  (Warmup)
## Chain 4 Iteration:  100 / 1500 [  6%]  (Warmup)
## Chain 3 Iteration:  100 / 1500 [  6%]  (Warmup)
## Chain 1 Iteration:  100 / 1500 [  6%]  (Warmup)
## Chain 1 Iteration:  200 / 1500 [ 13%]  (Warmup)
## Chain 1 Iteration:  300 / 1500 [ 20%]  (Warmup)
## Chain 1 Iteration:  400 / 1500 [ 26%]  (Warmup)
## Chain 1 Iteration:  500 / 1500 [ 33%]  (Warmup)
## Chain 1 Iteration:  501 / 1500 [ 33%]  (Sampling)
## Chain 4 Iteration:  200 / 1500 [ 13%]  (Warmup)
## Chain 3 Iteration:  200 / 1500 [ 13%]  (Warmup)
## Chain 1 Iteration:  600 / 1500 [ 40%]  (Sampling)
## Chain 4 Iteration:  300 / 1500 [ 20%]  (Warmup)
## Chain 4 Iteration:  400 / 1500 [ 26%]  (Warmup)
## Chain 1 Iteration:  700 / 1500 [ 46%]  (Sampling)
## Chain 3 Iteration:  300 / 1500 [ 20%]  (Warmup)
## Chain 2 Iteration:  200 / 1500 [ 13%]  (Warmup)
## Chain 1 Iteration:  800 / 1500 [ 53%]  (Sampling)
## Chain 3 Iteration:  400 / 1500 [ 26%]  (Warmup)
## Chain 4 Iteration:  500 / 1500 [ 33%]  (Warmup)
## Chain 4 Iteration:  501 / 1500 [ 33%]  (Sampling)
## Chain 3 Iteration:  500 / 1500 [ 33%]  (Warmup)
## Chain 1 Iteration:  900 / 1500 [ 60%]  (Sampling)
## Chain 3 Iteration:  501 / 1500 [ 33%]  (Sampling)
## Chain 4 Iteration:  600 / 1500 [ 40%]  (Sampling)
## Chain 3 Iteration:  600 / 1500 [ 40%]  (Sampling)
## Chain 1 Iteration: 1000 / 1500 [ 66%]  (Sampling)
## Chain 4 Iteration:  700 / 1500 [ 46%]  (Sampling)
## Chain 3 Iteration:  700 / 1500 [ 46%]  (Sampling)
## Chain 1 Iteration: 1100 / 1500 [ 73%]  (Sampling)
## Chain 4 Iteration:  800 / 1500 [ 53%]  (Sampling)
## Chain 3 Iteration:  800 / 1500 [ 53%]  (Sampling)
## Chain 1 Iteration: 1200 / 1500 [ 80%]  (Sampling)
## Chain 4 Iteration:  900 / 1500 [ 60%]  (Sampling)
## Chain 3 Iteration:  900 / 1500 [ 60%]  (Sampling)
## Chain 1 Iteration: 1300 / 1500 [ 86%]  (Sampling)
## Chain 4 Iteration: 1000 / 1500 [ 66%]  (Sampling)
## Chain 3 Iteration: 1000 / 1500 [ 66%]  (Sampling)
```

```
## Chain 1 Iteration: 1400 / 1500 [ 93%]  (Sampling)
## Chain 4 Iteration: 1100 / 1500 [ 73%]  (Sampling)
## Chain 3 Iteration: 1100 / 1500 [ 73%]  (Sampling)
## Chain 1 Iteration: 1500 / 1500 [100%]  (Sampling)
## Chain 1 finished in 87.2 seconds.
## Chain 4 Iteration: 1200 / 1500 [ 80%]  (Sampling)
## Chain 3 Iteration: 1200 / 1500 [ 80%]  (Sampling)
## Chain 4 Iteration: 1300 / 1500 [ 86%]  (Sampling)
## Chain 3 Iteration: 1300 / 1500 [ 86%]  (Sampling)
## Chain 4 Iteration: 1400 / 1500 [ 93%]  (Sampling)
## Chain 3 Iteration: 1400 / 1500 [ 93%]  (Sampling)
## Chain 2 Iteration:  300 / 1500 [ 20%]  (Warmup)
## Chain 4 Iteration: 1500 / 1500 [100%]  (Sampling)
## Chain 4 finished in 102.0 seconds.
## Chain 3 Iteration: 1500 / 1500 [100%]  (Sampling)
## Chain 3 finished in 103.6 seconds.
## Chain 2 Iteration:  400 / 1500 [ 26%]  (Warmup)
## Chain 2 Iteration:  500 / 1500 [ 33%]  (Warmup)
## Chain 2 Iteration:  501 / 1500 [ 33%]  (Sampling)
## Chain 2 Iteration:  600 / 1500 [ 40%]  (Sampling)
## Chain 2 Iteration:  700 / 1500 [ 46%]  (Sampling)
## Chain 2 Iteration:  800 / 1500 [ 53%]  (Sampling)
## Chain 2 Iteration:  900 / 1500 [ 60%]  (Sampling)
## Chain 2 Iteration: 1000 / 1500 [ 66%]  (Sampling)
## Chain 2 Iteration: 1100 / 1500 [ 73%]  (Sampling)
## Chain 2 Iteration: 1200 / 1500 [ 80%]  (Sampling)
## Chain 2 Iteration: 1300 / 1500 [ 86%]  (Sampling)
## Chain 2 Iteration: 1400 / 1500 [ 93%]  (Sampling)
## Chain 2 Iteration: 1500 / 1500 [100%]  (Sampling)
## Chain 2 finished in 155.6 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 112.1 seconds.
## Total execution time: 155.6 seconds.
```

```
prior_settings_m2 <- list(
  baseline = list(beta0_sd = 10, beta_sd = 1, tau_rate = 1, sigma_rate = 1),
  diffuse  = list(beta0_sd = 15, beta_sd = 1.5, tau_rate = 0.75, sigma_rate = 0.75),
  regular  = list(beta0_sd = 5, beta_sd = 0.5, tau_rate = 2, sigma_rate = 2)
)

fits_m2_sens <- lapply(prior_settings_m2, function(pr) {
  mod2_sens$sample(
    data = c(
      list(
        N = stan_data_model2$N,
        R = stan_data_model2$R,
        y = stan_data_model2$y,
        distance = stan_data_model2$distance,
        buildingarea = stan_data_model2$buildingarea,
        region = stan_data_model2$region
      ),
      pr
    ),
    chains = 4,
    parallel_chains = 4,
    iter_warmup = 500,
    iter_sampling = 1000,
    seed = 123,
    init = 0,
    refresh = 0,
    adapt_delta = 0.99,
    max_treedepth = 12
  )
})
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 finished in 181.5 seconds.
## Chain 4 finished in 224.8 seconds.
## Chain 3 finished in 336.6 seconds.
## Chain 2 finished in 1341.8 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 521.2 seconds.
## Total execution time: 1342.0 seconds.
##
## Running MCMC with 4 parallel chains...
##
## Chain 4 finished in 168.3 seconds.
## Chain 1 finished in 178.6 seconds.
## Chain 3 finished in 253.1 seconds.
## Chain 2 finished in 1545.9 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 536.5 seconds.
## Total execution time: 1546.0 seconds.
##
## Running MCMC with 4 parallel chains...
##
## Chain 4 finished in 123.8 seconds.
## Chain 1 finished in 243.3 seconds.
## Chain 2 finished in 348.5 seconds.
## Chain 3 finished in 393.4 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 277.2 seconds.
## Total execution time: 393.5 seconds.
```

```
summarize_fit_m2 <- function(fit, prior_name) {
  draws <- fit$draws(
    variables = c("beta0", "beta1", "beta2", "tau", "sigma"),
    format = "draws_df"
  )

  dplyr::bind_rows(
    data.frame(
      prior = prior_name,
      parameter = "beta0",
      mean = mean(draws$beta0),
      q2.5 = quantile(draws$beta0, 0.025),
      q97.5 = quantile(draws$beta0, 0.975)
    ),
    data.frame(
      prior = prior_name,
      parameter = "beta1",
      mean = mean(draws$beta1),
      q2.5 = quantile(draws$beta1, 0.025),
      q97.5 = quantile(draws$beta1, 0.975)
    ),
    data.frame(
      prior = prior_name,
      parameter = "beta2",
      mean = mean(draws$beta2),
      q2.5 = quantile(draws$beta2, 0.025),
      q97.5 = quantile(draws$beta2, 0.975)
    ),
    data.frame(
      prior = prior_name,
      parameter = "tau",
      mean = mean(draws$tau),
      q2.5 = quantile(draws$tau, 0.025),
      q97.5 = quantile(draws$tau, 0.975)
    ),
    data.frame(
      prior = prior_name,
      parameter = "sigma",
      mean = mean(draws$sigma),
      q2.5 = quantile(draws$sigma, 0.025),
      q97.5 = quantile(draws$sigma, 0.975)
    )
  )
}
```

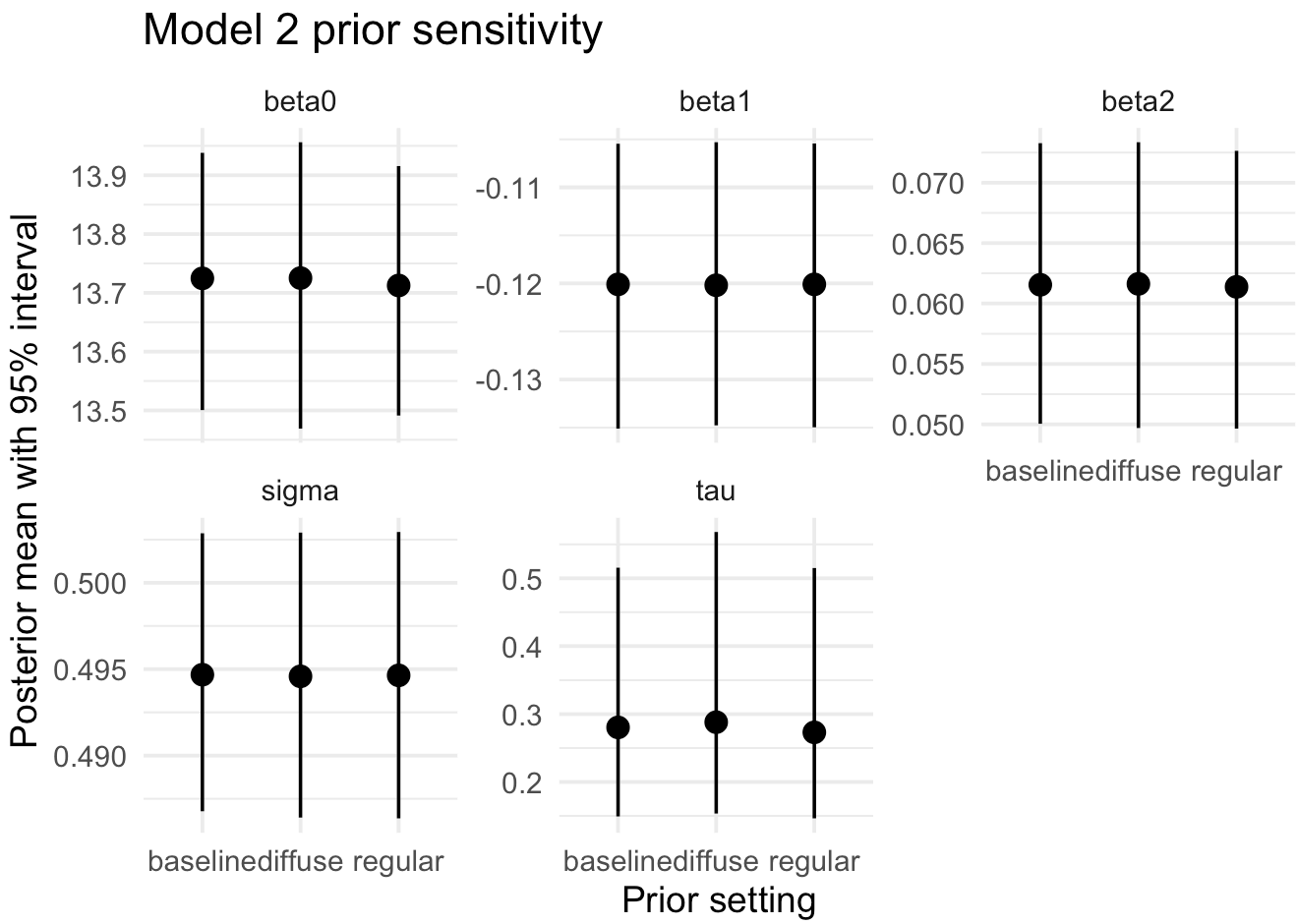
Sensitivity table

```
sens_table_m2 <- dplyr::bind_rows(
  summarize_fit_m2(fits_m2_sens$baseline, "baseline"),
  summarize_fit_m2(fits_m2_sens$diffuse, "diffuse"),
  summarize_fit_m2(fits_m2_sens$regular, "regular")
)

sens_table_m2
```

```
##              prior parameter        mean        q2.5       q97.5
## 2.5%...1  baseline     beta0 13.72464736 13.50042130 13.93832100
## 2.5%...2  baseline     beta1 -0.12009394 -0.13510736 -0.10546127
## 2.5%...3  baseline     beta2  0.06155228  0.05004539  0.07328031
## 2.5%...4  baseline       tau  0.28034265  0.14911178  0.51574218
## 2.5%...5  baseline     sigma  0.49468025  0.48677532  0.50286587
## 2.5%...6   diffuse     beta0 13.72510980 13.46885470 13.95636018
## 2.5%...7   diffuse     beta1 -0.12018715 -0.13476893 -0.10530218
## 2.5%...8   diffuse     beta2  0.06162718  0.04969279  0.07335686
## 2.5%...9   diffuse       tau  0.28803978  0.15340717  0.56830286
## 2.5%...10  diffuse     sigma  0.49458506  0.48641006  0.50290500
## 2.5%...11  regular     beta0 13.71249951 13.49110585 13.91574940
## 2.5%...12  regular     beta1 -0.12009336 -0.13497150 -0.10543584
## 2.5%...13  regular     beta2  0.06138108  0.04964292  0.07264699
## 2.5%...14  regular       tau  0.27305524  0.14627873  0.51519863
## 2.5%...15  regular     sigma  0.49464116  0.48636425  0.50293930
```

Graph

```
ggplot(sens_table_m2, aes(x = prior, y = mean, ymin = q2.5, ymax = q97.5)) +
  geom_pointrange() +
  facet_wrap(~ parameter, scales = "free_y") +
  labs(
    title = "Model 2 prior sensitivity",
    x = "Prior setting",
    y = "Posterior mean with 95% interval"
  ) +
  theme_minimal(base_size = 14)
```

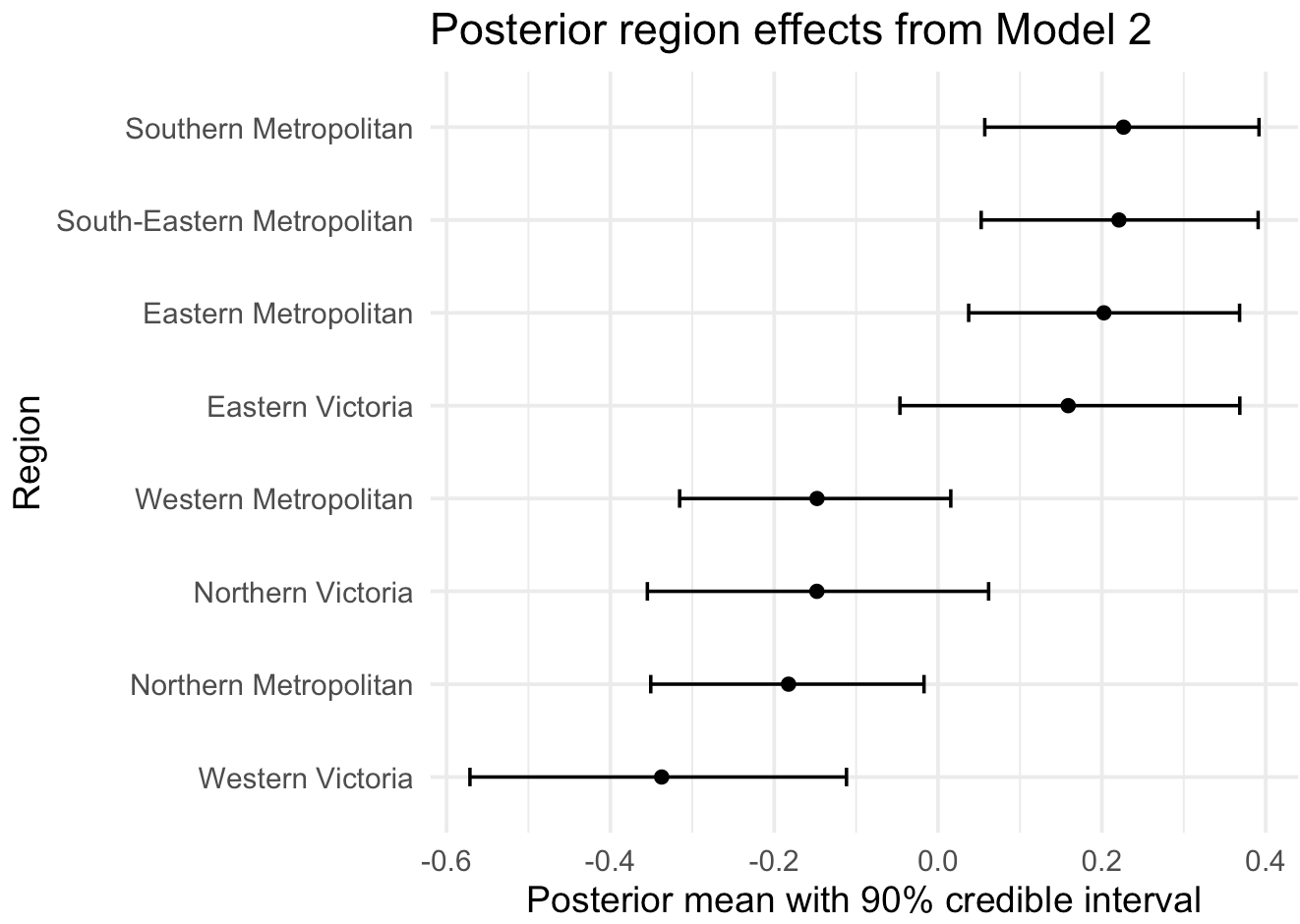


Regional effects from model 2 - interpetation

```
region_levels <- levels(as.factor(part1_data$Regionname))

u_summary <- fit_m2$summary(variables = "u") %>%
  as.data.frame() %>%
  mutate(region = region_levels) %>%
  dplyr::select(region, mean, median, sd, q5, q95, rhat, ess_bulk, ess_tail)

u_summary
```

```
##                        region       mean     median        sd          q5
## 1       Eastern Metropolitan  0.2024287  0.2044740 0.1028277  0.03734691
## 2           Eastern Victoria  0.1588666  0.1584612 0.1265924 -0.04656683
## 3      Northern Metropolitan -0.1825104 -0.1800906 0.1030669 -0.35064599
## 4          Northern Victoria -0.1479182 -0.1482832 0.1294015 -0.35483717
## 5 South-Eastern Metropolitan  0.2207947  0.2228247 0.1062008  0.05268376
## 6      Southern Metropolitan  0.2263961  0.2301013 0.1026183  0.05701544
## 7       Western Metropolitan -0.1477365 -0.1456615 0.1025257 -0.31546552
## 8           Western Victoria -0.3373575 -0.3333516 0.1391738 -0.57137219
##           q95     rhat  ess_bulk ess_tail
## 1  0.36808282 1.004679  813.6805 1255.912
## 2  0.36836832 1.004275 1233.3667 1927.270
## 3 -0.01712302 1.004901  811.8250 1216.422
## 4  0.06168870 1.001997 1190.1104 1873.767
## 5  0.39068849 1.003948  868.4004 1325.281
## 6  0.39172315 1.004796  807.5969 1267.065
## 7  0.01556407 1.005138  808.2858 1241.502
## 8 -0.11170570 1.002969 1294.0036 2218.089
```

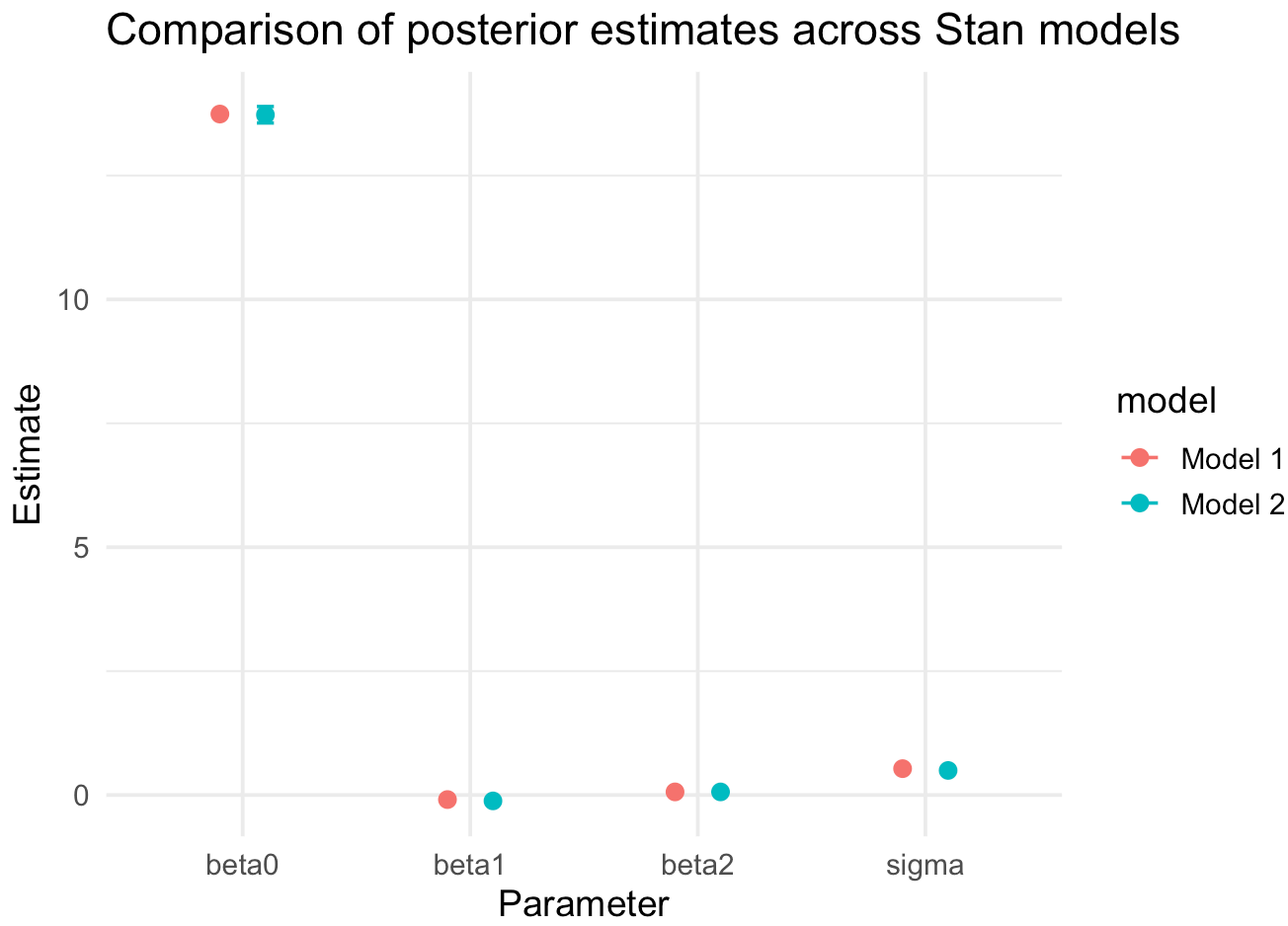
```
ggplot(u_summary, aes(x = reorder(region, mean), y = mean)) +
  geom_point() +
  geom_errorbar(aes(ymin = q5, ymax = q95), width = 0.2) +
  coord_flip() +
  labs(title = "Posterior region effects from Model 2",
       x = "Region", y = "Posterior mean with 90% credible interval") +
  theme_minimal(base_size = 14)
```



Model 1 vs Model 2

compare interval across model

```
compare_plot_df <- dplyr::bind_rows(
  summ_m1 %>% mutate(model = "Model 1"),
  summ_m2 %>% mutate(model = "Model 2")
) %>%
  filter(variable %in% c("beta0", "beta1", "beta2", "sigma"))

ggplot(compare_plot_df, aes(x = variable, y = mean, colour = model)) +
  geom_point(position = position_dodge(width = 0.4), size = 2.5) +
  geom_errorbar(aes(ymin = q5, ymax = q95), position = position_dodge(width = 0.4), widt
h = 0.15) +
  labs(title = "Comparison of posterior estimates across Stan models",
       x = "Parameter", y = "Estimate") +
  theme_minimal(base_size = 14)
```



Model adequacy and predictive comparison

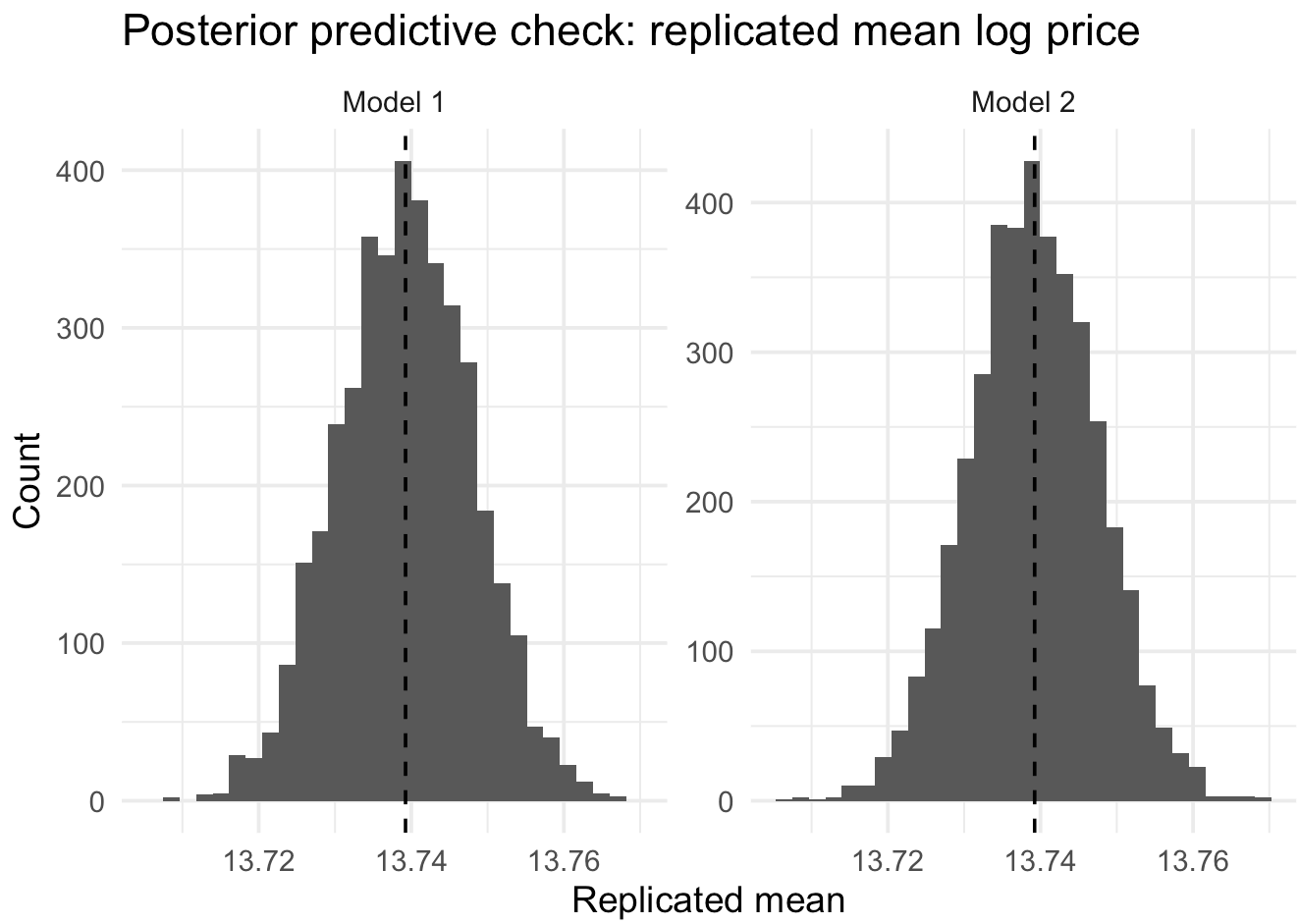
Note: All of the following is a posterior predictive diagnostics

Extract posterior draws

```
yrep_m1 <- fit_m1$draws("y_rep", format = "matrix")
yrep_m2 <- fit_m2$draws("y_rep", format = "matrix")

y_obs <- stan_data_model1$y
```

Diagnostics - PPC

```
ppc_df <- dplyr::bind_rows(
  data.frame(model = "Model 1", y_rep_mean = rowMeans(yrep_m1)),
  data.frame(model = "Model 2", y_rep_mean = rowMeans(yrep_m2))
)

obs_mean <- mean(y_obs)

ggplot(ppc_df, aes(x = y_rep_mean)) +
  geom_histogram(bins = 30) +
  geom_vline(xintercept = obs_mean, linetype = 2) +
  facet_wrap(~ model, scales = "free_y") +
  labs(title = "Posterior predictive check: replicated mean log price",
       x = "Replicated mean", y = "Count") +
  theme_minimal(base_size = 14)
```

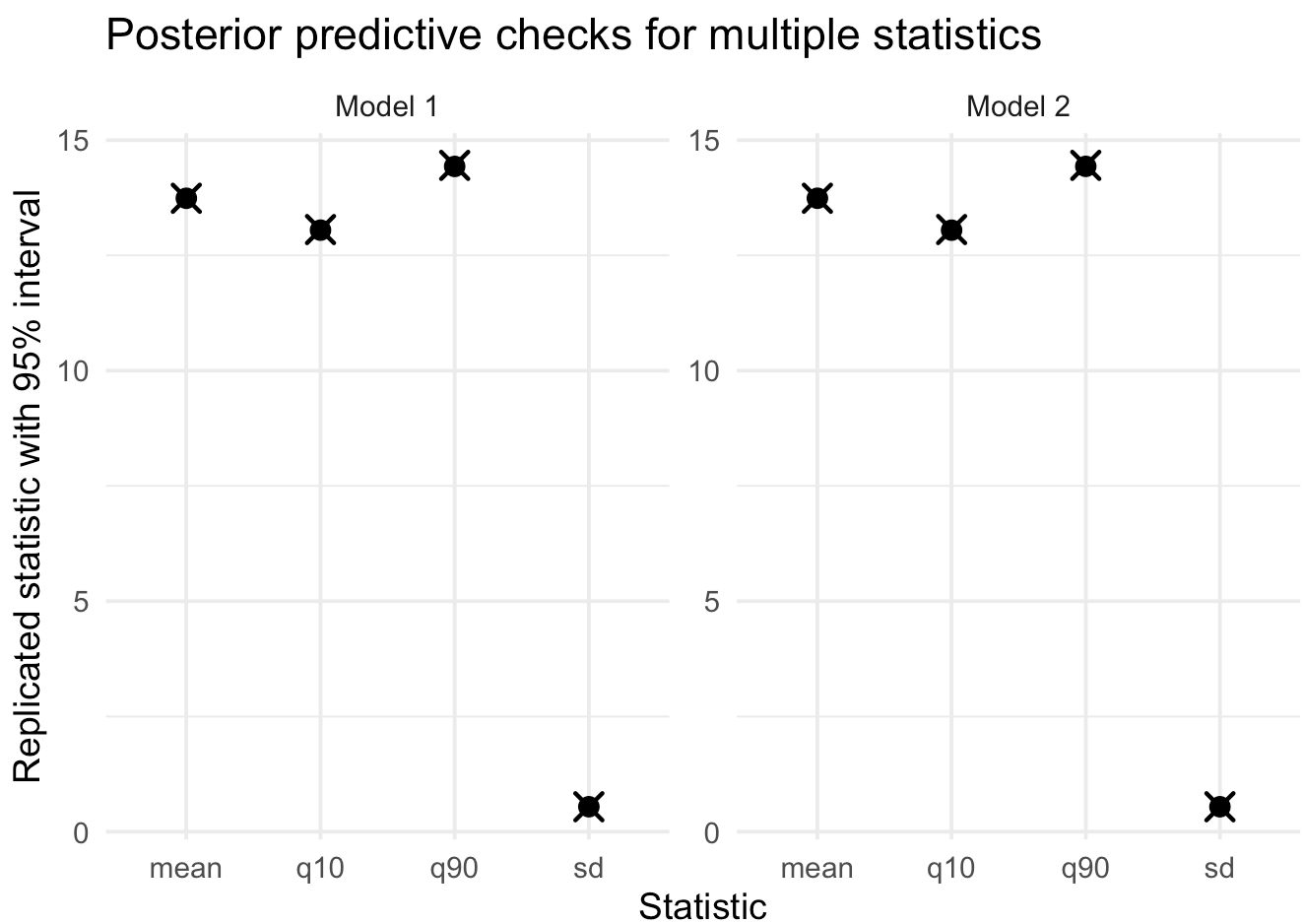


Summary statistics for PCC

```
ppc_stats <- function(yrep, y_obs, model_name) {
  data.frame(
    model = model_name,
    stat = c("mean", "sd", "q10", "q90"),
    observed = c(
      mean(y_obs),
      sd(y_obs),
      quantile(y_obs, 0.10),
      quantile(y_obs, 0.90)
    ),
    rep_mean = c(
      mean(rowMeans(yrep)),
      mean(apply(yrep, 1, sd)),
      mean(apply(yrep, 1, quantile, probs = 0.10)),
      mean(apply(yrep, 1, quantile, probs = 0.90))
    ),
    lower = c(
      quantile(rowMeans(yrep), 0.025),
      quantile(apply(yrep, 1, sd), 0.025),
      quantile(apply(yrep, 1, quantile, probs = 0.10), 0.025),
      quantile(apply(yrep, 1, quantile, probs = 0.90), 0.025)
    ),
    upper = c(
      quantile(rowMeans(yrep), 0.975),
      quantile(apply(yrep, 1, sd), 0.975),
      quantile(apply(yrep, 1, quantile, probs = 0.10), 0.975),
      quantile(apply(yrep, 1, quantile, probs = 0.90), 0.975)
    )
  )
}

ppc_stats_all <- dplyr::bind_rows(
  ppc_stats(yrep_m1, y_obs, "Model 1"),
  ppc_stats(yrep_m2, y_obs, "Model 2")
)

ppc_stats_all
```

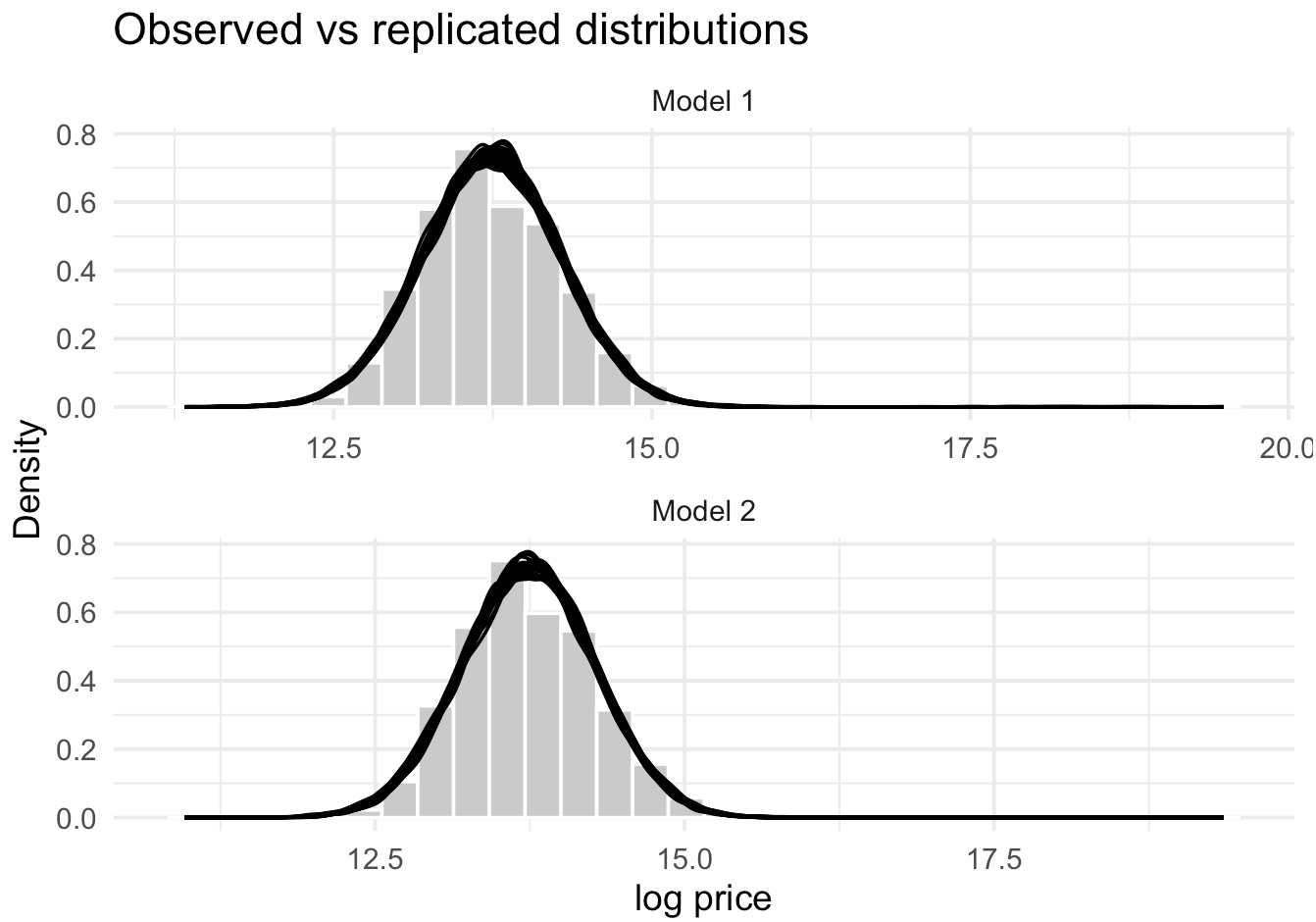
```
##     model stat   observed   rep_mean      lower      upper
## 1 Model 1 mean 13.7392423 13.7392250 13.7222457 13.7565747
## 2 Model 1   sd  0.5418189  0.5420111  0.5298972  0.5547856
## 3 Model 1  q10 13.0604880 13.0475141 13.0205295 13.0741994
## 4 Model 1  q90 14.4521403 14.4294607 14.4029939 14.4562921
## 5 Model 2 mean 13.7392423 13.7391214 13.7225854 13.7556621
## 6 Model 2   sd  0.5418189  0.5420115  0.5301467  0.5541927
## 7 Model 2  q10 13.0604880 13.0464883 13.0208072 13.0726904
## 8 Model 2  q90 14.4521403 14.4302187 14.4040696 14.4559088
```

```
ggplot(ppc_stats_all, aes(x = stat, y = rep_mean)) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = lower, ymax = upper), width = 0.15) +
  geom_point(aes(y = observed), shape = 4, size = 4, stroke = 1.2) +
  facet_wrap(~ model, scales = "free_y") +
  labs(title = "Posterior predictive checks for multiple statistics",
       x = "Statistic", y = "Replicated statistic with 95% interval") +
  theme_minimal(base_size = 14)
```



Observed vs replicated distribution

```
set.seed(123)

draw_ids_m1 <- sample(1:nrow(yrep_m1), 30)
draw_ids_m2 <- sample(1:nrow(yrep_m2), 30)

rep_df_m1 <- data.frame(
  value = as.vector(t(yrep_m1[draw_ids_m1, ])),
  draw = rep(1:30, each = ncol(yrep_m1)),
  model = "Model 1"
)

rep_df_m2 <- data.frame(
  value = as.vector(t(yrep_m2[draw_ids_m2, ])),
  draw = rep(1:30, each = ncol(yrep_m2)),
  model = "Model 2"
)

rep_df_all <- dplyr::bind_rows(rep_df_m1, rep_df_m2)
obs_df <- data.frame(value = y_obs)

ggplot() +
  geom_histogram(data = obs_df, aes(x = value, y = after_stat(density)),
                 bins = 30, fill = "grey80", color = "white") +
  geom_density(data = rep_df_all, aes(x = value, group = interaction(model, draw)),
               alpha = 0.12) +
  facet_wrap(~ model, scales = "free", ncol = 1) +
  labs(title = "Observed vs replicated distributions",
       x = "log price", y = "Density") +
  theme_minimal(base_size = 14)
```



Predictive adequacy check

LOO comparison

```
loglik_m1 <- fit_m1$draws("log_lik", format = "matrix")
loglik_m2 <- fit_m2$draws("log_lik", format = "matrix")

loo_m1 <- loo::loo(loglik_m1)
```

```
## Warning: Some Pareto k diagnostic values are too high. See help('pareto-k-diagnosti
c') for details.
```

```
loo_m2 <- loo::loo(loglik_m2)
```

```
## Warning: Some Pareto k diagnostic values are too high. See help('pareto-k-diagnosti
c') for details.
```

```
print(loo_m1)
```

```
##
## Computed from 4000 by 7130 log-likelihood matrix.
##
##          Estimate    SE
## elpd_loo  -5630.1  80.9
## p_loo        43.2  40.5
## looic     11260.2 161.9
## ------
## MCSE of elpd_loo is NA.
## MCSE and ESS estimates assume independent draws (r_eff=1).
##
## Pareto k diagnostic values:
##                          Count Pct.    Min. ESS
## (-Inf, 0.7]   (good)     7129  100.0%  3860
##    (0.7, 1]   (bad)         0    0.0%  <NA>
##    (1, Inf)   (very bad)    1    0.0%  <NA>
## See help('pareto-k-diagnostic') for details.
```

```
print(loo_m2)
```

```
##
## Computed from 4000 by 7130 log-likelihood matrix.
##
##          Estimate    SE
## elpd_loo  -5124.8  81.8
## p_loo        42.8  35.9
## looic     10249.7 163.7
## ------
## MCSE of elpd_loo is NA.
## MCSE and ESS estimates assume independent draws (r_eff=1).
##
## Pareto k diagnostic values:
##                          Count Pct.    Min. ESS
## (-Inf, 0.7]   (good)     7129  100.0%  3512
##    (0.7, 1]   (bad)         0    0.0%  <NA>
##    (1, Inf)   (very bad)    1    0.0%  <NA>
## See help('pareto-k-diagnostic') for details.
```

```
print(loo_compare(loo_m1, loo_m2))
```

```
##        elpd_diff se_diff
## model2    0.0       0.0
## model1 -505.3      31.5
```